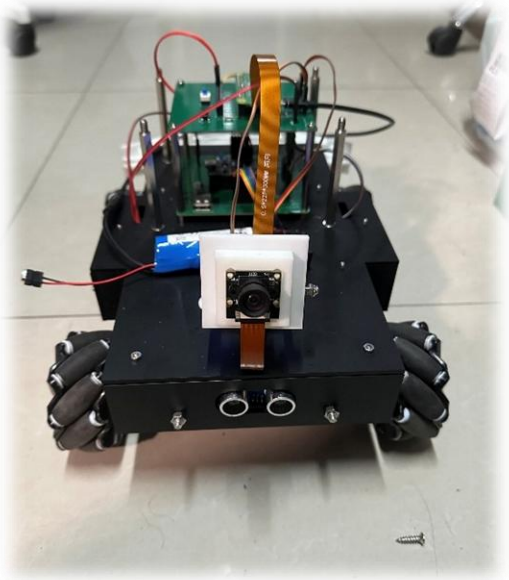
中南大學 邓迪国际学院
DUNDEE INTERNATIONAL INSTITUTE OF CENTRAL SOUTH UNIVERSITY

DI31013 – Engineering Design Project

Multifunctional Vision Car

Mechanical Engineering

Group 15



Zhiyi Zhu	Ruiyang Zhao	Liang Chen	Jinglin Li	Chenchenjie Liu	Chenyu Ma
2617533	2617516	2617444	2617384	2617339	2617613
Software Development	PCB Design	Microcontroller Implementation	Mechanical System Analysis	Experiments and function tests	Mechanical Design

中南大学 邓迪国际学院
DUNDEE INTERNATIONAL INSTITUTE OF CENTRAL SOUTH UNIVERSITY

Content

Abstract **1**

1. Introduction **1**

2. Mechanical Design **2**

2.1. Design Overview 2
2.2. Material Selection and Manufacturing 2
2.3. Structural Evolution and Iterations 3
2.4. Impact Protection and Bumper Design 4
2.5. Real-Time Monitoring System 4
2.6. Assembly and Connection Details 5

3. Mechanical System Analysis **6**

3.1. Explicit Dynamic Stress Analysis (ANSYS Explicit Dynamics) 6
3.2. Static Stress Analysis (SolidWorks Simulation) 9
3.3. Static – Dynamic Analysis Comparison and Engineering Significance 11
3.4. Conclusions and Suggestions for Improvement 11

4. Microcontroller Implementation **12**

5. Electrical Circuit and PCB Design **15**

5.1. LPC1768 Motion Control Subsystem 15
5.2. Raspberry Pi Implementation and Design Iteration 16
5.3. PCB Design 17
5.4. Physical PCB layouts 18
5.5. PCB Fabrication 19

6. Software Development **21**

6.1. Motion Control: LPC1768 21
6.2. Vision: Raspberry Pi Zero 2 W 26
6.3. User Interaction: WeChat Mini Program 31

7. Experiments and Results **33**

8. Lessons Learned **35**

9. Future Work **36**

10. Conclusion **37**

References **37**

Appendix ..... **I**

A. Command Manual I

B. Crucial Code I

B.1. Motion Control (LPC1768) ..... *I*

B.2. Vision Module (Raspberry Pi) ..... *XI*

B.3. WeChat Mini Program ..... *XXI*

Abstract

This report details the design and implementation of an omnidirectional mobile robotic vehicle system integrating mechanical engineering design with real-time visual capabilities. Regarding mechanical design, the chassis employs a modular structure constructed from high-strength acrylic panels, significantly enhancing the system's load capacity and stability. The mobility system comprises four independently driven Mecanum wheels arranged in an “X” configuration, enabling omnidirectional movement including lateral translation and rotation. Furthermore, the system incorporates a custom-designed two-degree-of-freedom (translation-pitch) servo gimbal to provide active visual tracking capabilities.

To support this hardware, the system employs a distributed dual-controller architecture: the LPC1768 microcontroller manages chassis movement and motor control, while the Raspberry Pi Zero 2 W handles high-bandwidth image processing. The software implementation employs a hybrid concurrency model combining Python asyncio and multithreading, ensuring low-latency video streaming via WebSockets. Additionally, a WeChat Mini Program was developed for the user interface, utilizing a dual-channel protocol to achieve low-latency virtual joystick control. This design facilitates efficient synergy between mechanical and software components, delivering a stable and highly interactive operating platform.

1. Introduction

In recent years, traditional manual border patrol methods have shown many limitations in practical applications. Patrol tasks usually require high labor intensity and often need to be carried out in harsh environments, which may pose safety risks to personnel. In addition, manual patrol has limited coverage and relatively slow response to potential threats, making it difficult to meet the requirements of modern border monitoring. Therefore, intelligent and automated border patrol systems have attracted increasing attention [1]-[2]. Unmanned systems with sensing and information processing capabilities provide an effective way to improve patrol efficiency and safety [1].

Although unmanned aerial vehicles have been widely used in monitoring tasks, especially in visual perception and real-time information processing [1]-[2], ground unmanned patrol platforms still face several challenges in real environments. Problems such as limited mobility on complex terrain, insufficient integration of sensing modules, and unstable multi-sensor cooperation are still common. As a result, many existing grounds patrol systems cannot achieve reliable and long-term operation.

To address these issues, this paper designs a lightweight intelligent border patrol vehicle based on the NXP LPC1768 embedded platform, with software developed in the Mbed Studio environment. The system integrates Bluetooth remote control, omni-directional movement, and ultrasonic obstacle avoidance functions. A Raspberry Pi Zero 2W with an IMX219 camera is used for image acquisition and visual processing, while the LPC1768 microcontroller handles motion control and sensor coordination, improving the overall stability of the system.

To verify the feasibility of the proposed design, simulation analyses were carried out. ANSYS was used to analyze the motion characteristics and dynamic response of the vehicle under typical working conditions, while SolidWorks was applied to evaluate the stress and deformation of the chassis structure. The results show that the patrol vehicle has reasonable structural strength and stable motion performance, providing support for its practical application.

In summary, the designed patrol vehicle achieves basic functions such as patrol, detection, recording, and feedback. With its advantages in integration, cost, and ease of deployment, the system provides a useful reference for the development of ground unmanned patrol platforms in border monitoring applications.

2. Mechanical Design

2.1. Design Overview

The mechanical design of this project aims to construct a versatile and adaptable border patrol vehicle, with the overall appearance shown in Figure 1. The system integrates Bluetooth remote control functionality, emergency obstacle avoidance capabilities and is equipped with a real-time video surveillance system. The chassis employs a quad-motor drive configuration with Mecanum wheels, enabling omnidirectional mobility [3].

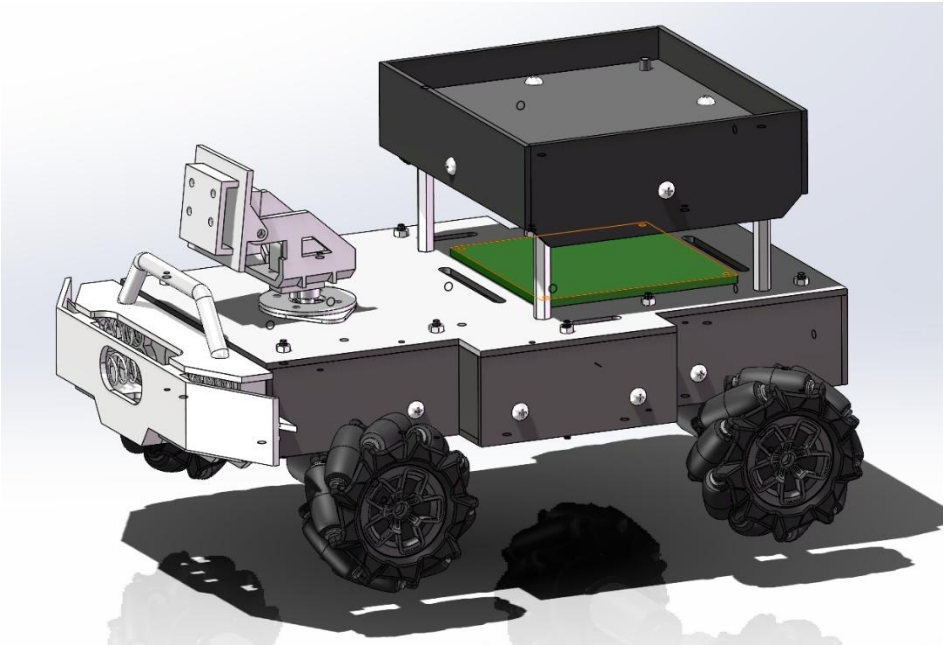


Figure 1 Exterior of the vehicle.

2.2. Material Selection and Manufacturing

The vehicle body utilizes acrylic sheet as the primary structural material, fabricated using high-precision laser cutting techniques. This manufacturing process achieves a dimensional tolerance of approximately 0.1 mm,

ensuring a precise fit between motor mounting interfaces and bearing components, which is critical for accurate directional control. Compared to metallic alternatives, acrylic offers a superior strength-to-weight ratio[4]. This lightweight characteristic significantly reduces the vehicle's unladen weight, thereby alleviating motor load and optimizing battery energy efficiency. Simultaneously, the material maintains sufficient structural rigidity to prevent chassis distortion during dynamic manoeuvres. Furthermore, the inherent electrical insulating properties of acrylic effectively eliminate the risk of short circuits within the mounted electronic modules.

To enhance the reliability of the mechanical structure, specific optimization schemes were implemented for the connection hardware and transmission system. Stainless steel fasteners are employed throughout the chassis to ensure robustness and corrosion resistance. At critical structural nodes subject to high stress concentrations, reinforced metal connectors are utilized to prevent loosening or fatigue failure resulting from prolonged operation. For the propulsion system, the Mecanum wheels feature hubs constructed from high-strength ABS plastic to provide impact resistance, while the rollers are composed of rubber. This material combination ensures excellent traction and smooth motion without slippage, which is essential for achieving precise omnidirectional mobility.

2.3. Structural Evolution and Iterations

Based on empirical data obtained during the development process, the mechanical structure evolved through two primary design iterations. The initial phase utilized a straightforward double-layer sandwich configuration. However, operational testing revealed a critical deficiency: during rapid acceleration or abrupt deceleration, internal components experienced significant inertial displacement. This movement sharply increased the risk of cable disconnection, leading to potential power loss or signal failure that rendered the vehicle inoperable.

To mitigate these risks, the internal chassis was redesigned into a compartmentalized grid structure, as illustrated in Figure 2. In this final design, vertical acrylic partitions are introduced to create distinct housing bays for key components, including the battery, motor driver, and main control module. This optimization serves a dual purpose: the physical barriers effectively constrain component movement, preventing circuit interruptions, while the vertical partitions function as internal stiffeners. Consequently, this design significantly enhances the torsional rigidity of the chassis compared to the initial flat-plate configuration.

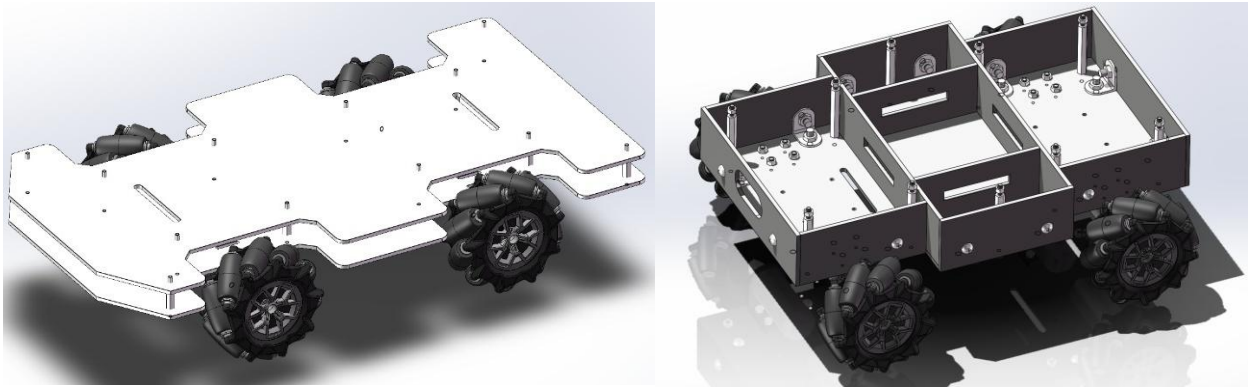


Figure 2 General view of double-layer design and compartmentalized chassis design.

2.4. Impact Protection and Bumper Design

To enhance the vehicle's collision resistance, a spring-loaded buffer mechanism was incorporated into the front-end design, as illustrated in Figure 3. This system is designed to absorb most of the impact energy during frontal collisions, thereby maintaining chassis stability and safeguarding the delicate ultrasonic sensors from direct physical damage. Structurally, the bumper is strengthened with additional vertical reinforcing ribs, which provide the necessary rigidity to prevent bending or cracking under stress. Furthermore, the design features dedicated recesses that allow the ultrasonic sensors to sit flush with the vehicle's front surface. This flush-mounting configuration is critical for avoiding signal interference caused by chassis edge diffraction, ensuring accurate sound wave propagation and reliable obstacle detection.

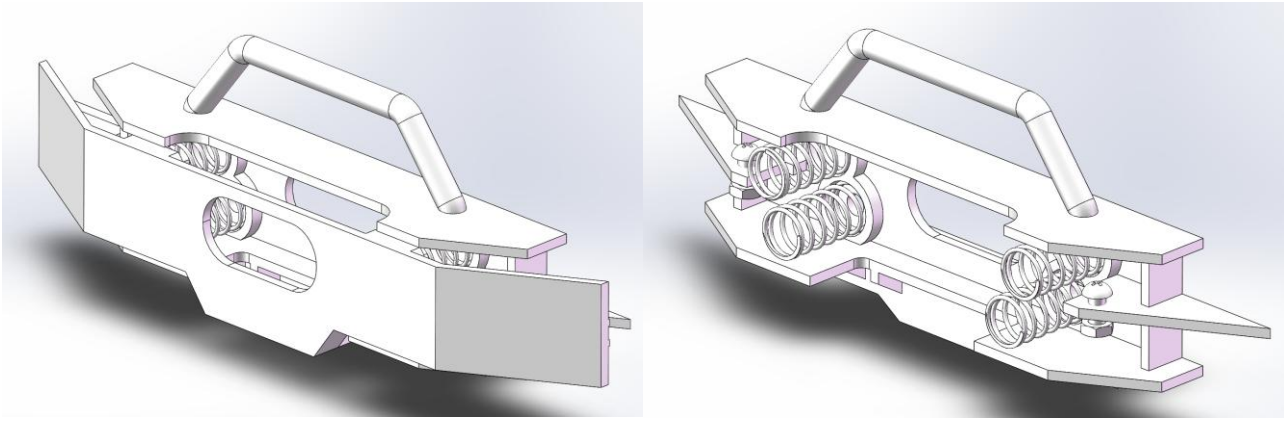


Figure 3 Bumper detail diagram.

2.5. Real-Time Monitoring System

This design presents a two-degree-of-freedom gimbal mechanical structure (Figure 4) dedicated to vision systems, aiming to integrate two servos and a camera module through precise mechanical linkages. The structure comprises a rotating base, a U-shaped cantilever bracket, and a front mounting panel. The bottom servo drives the bracket to perform horizontal scanning, while the side servo actuates the panel to achieve

vertical pitch adjustment, thereby endowing the camera with a flexible spatial viewing angle.

In terms of structural optimization, triangular truss-like cutouts are adopted on both sides of the cantilever. This lightweight design strategy significantly reduces the mass and moment of inertia of the moving parts while maintaining structural rigidity and torsional strength, effectively alleviating the load on the servos. Simultaneously, the compact hinge support structure effectively eliminates mechanical backlash, ensuring the stability of the camera footage during vehicle locomotion or rapid maneuvering.

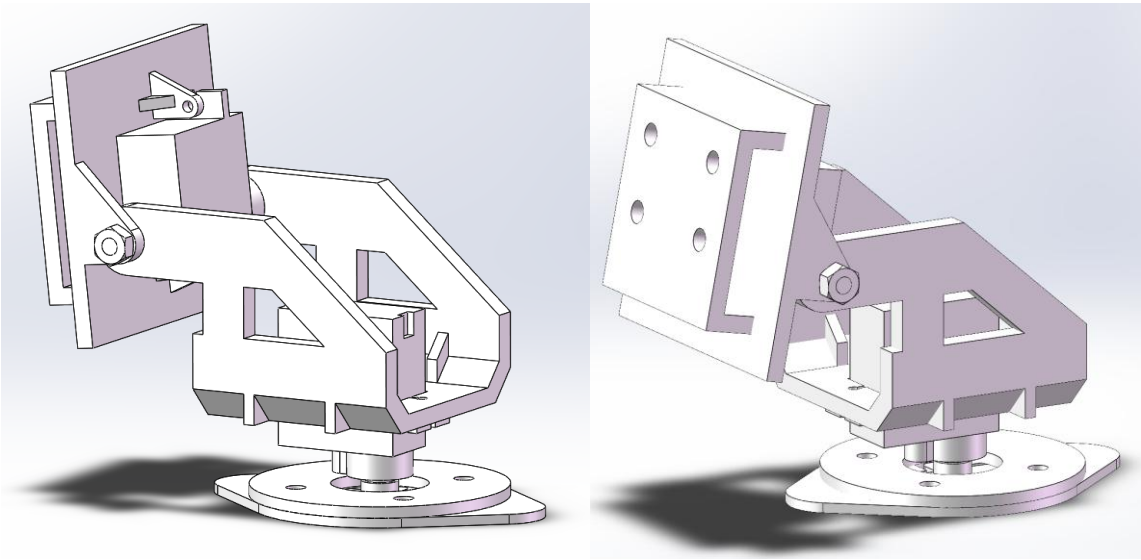


Figure 4 The two-degree-of-freedom gimbal system.

2.6. Assembly and Connection Details

To ensure structural integrity under dynamic operation and vibration conditions, a multi-tiered precision assembly scheme was implemented.

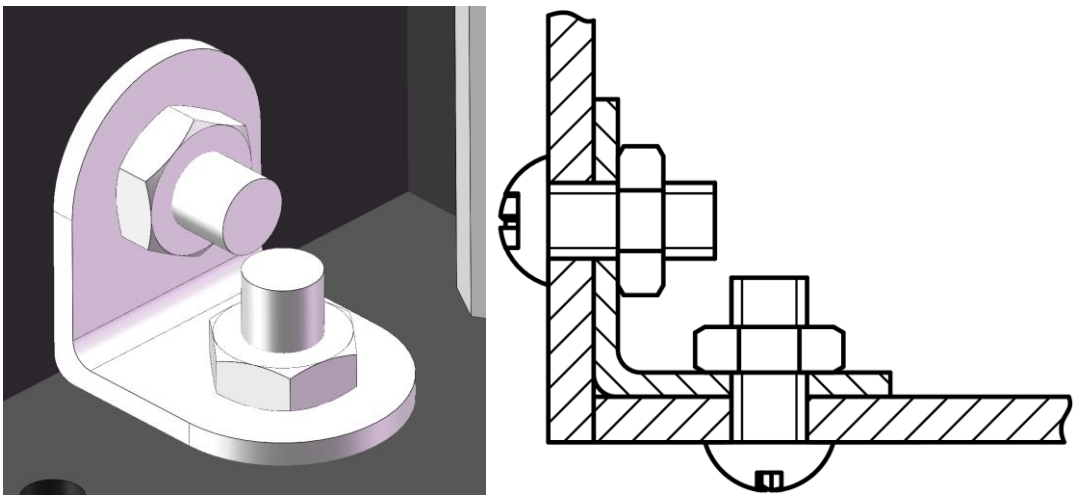


Figure 5 L-bracket Diagram of 3D model and assembly section view.

For the primary chassis framework, metal L-brackets were employed to lock the vertical panels to the base plate, reinforcing the initial slot positioning as illustrated in Figure 5. This dual-fixing strategy constructs a robust three-dimensional load-bearing grid. Furthermore, to maintain the parallelism and stability of the multi-level architecture, the upper deck is anchored to the chassis via metal double-headed hexagonal support studs (Figure 6). This through-type connection ensures a stable and rigid interface between the upper and lower deck structures.

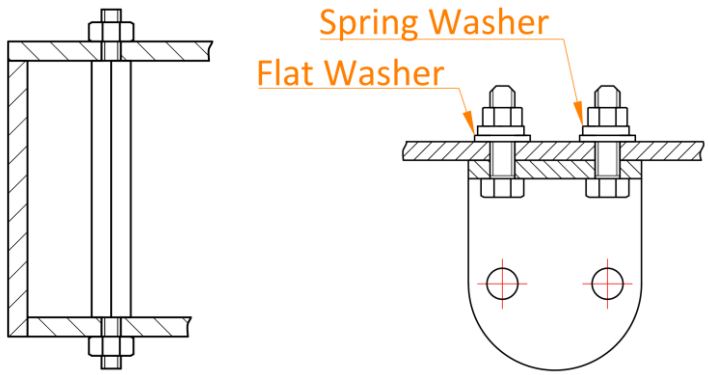


Figure 6 Assembly diagram of top boards and base boards.

Figure 7 Assembly diagram of base board and connector of motor

Addressing the motor as the primary source of operational vibration, specific seismic and anti-loosening measures were introduced at critical connection points (Figure 7). Spring washers positioned between the mounting nuts and the base plate serve a dual purpose: they function as mechanical dampers to filter high-frequency vibrations generated by the motor, and when paired with lock nuts, the axial tension provided by the spring washers effectively prevents fastener loosening induced by prolonged vibration, thereby ensuring the long-term reliability of the drive system [5].

3. Mechanical System Analysis

3.1. Explicit Dynamic Stress Analysis (ANSYS Explicit Dynamics)

To investigate the transient dynamic response of the vehicle under frontal collision conditions, an explicit dynamic finite element analysis was carried out using ANSYS Explicit Dynamics. Considering the limitations of computational resources and solution efficiency, the original vehicle model was simplified on the premise that the overall dynamic characteristics were preserved (Figure 8). Only the vehicle body and the front bumper were retained in the model, while components with limited influence on the collision response were omitted. During the simplification process, the total mass, mass distribution, and centroid position of the simplified model were adjusted to be approximately consistent with those of the original structure by modifying material density or introducing equivalent lumped mass. This treatment ensures that the inertial response during collision remains comparable.

Based on theoretical estimation of the driving system, the vehicle speed under ideal no-load conditions can reach approximately 1.6 m/s after considering the velocity decomposition caused by the Mecanum wheel motion. However, in practical operation, the actual speed is significantly reduced due to factors such as friction, wheel slip, payload, and control strategy. Therefore, to better represent realistic low-speed collision conditions while maintaining numerical stability, the initial velocity in the explicit dynamic analysis was set to 1.0 m/s. The initial distance between the vehicle and the rigid wall was defined as 80 mm, and the total simulation time was set to 0.1 s, which is sufficient to cover contact initiation, deformation development, and energy dissipation stages.

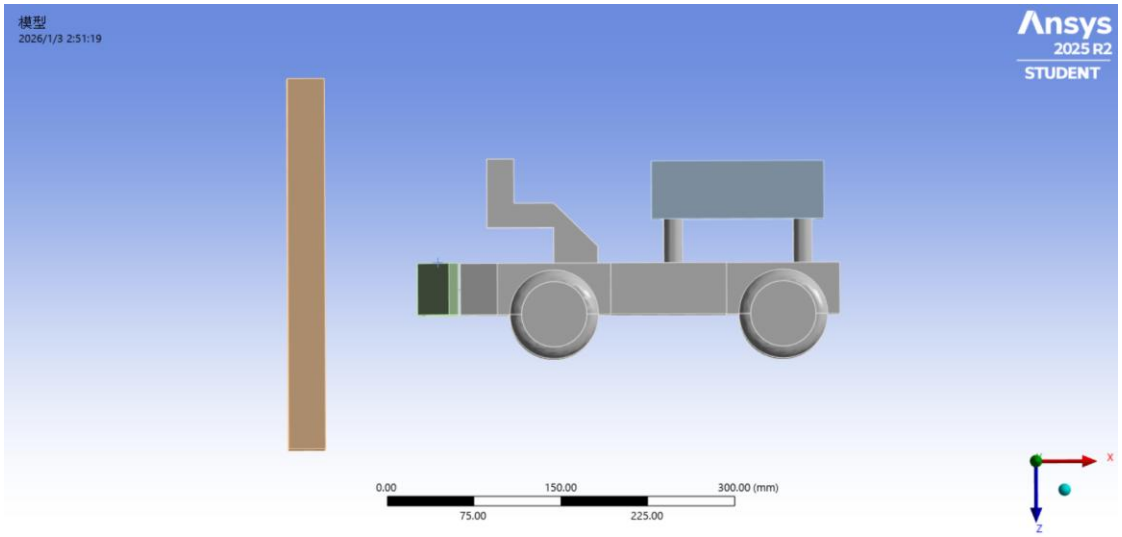


Figure 8 Schematic of the explicit dynamic analysis model and collision condition

In terms of material modeling, both the vehicle body and the front bumper were assumed to be made of ABS material and described using an isotropic linear elastic constitutive model. This assumption was adopted to focus on the elastic response characteristics and stress distribution trends under low-speed collision conditions, without considering material yielding, damage, or fracture. The rigid obstacle was modeled using structural steel and discretized with three layers of solid elements to provide sufficient stiffness as a load-transmitting boundary.

To model the energy absorption behavior of the front anti-collision structure, the four springs used in the original design were simplified into an equivalent single spring–damper element. Assuming that the springs are connected in parallel, the equivalent stiffness and damping coefficients can be expressed as:

$$k_{eq} = \sum_{i=1}^{4} k_i \, , c_{eq} = \sum_{i=1}^{4} c_i \tag{1}$$

Based on this equivalence, the stiffness and damping of the simplified element were set to $k_{eq} = 3.2$ N/mm and $c_{eq} = 0.12$ Ns/mm, respectively. This simplification reduces the computational complexity of the model while maintaining comparable stiffness characteristics and energy dissipation behavior of the original front

buffering system.

Simulation results show that during the initial stage of collision, the front bumper first meets the rigid wall and undergoes obvious transient deformation (Figure 9). As the primary impact component, the bumper experiences significantly higher stress levels than the vehicle body, indicating that it bears the main impact load. Meanwhile, the equivalent spring–damper system converts part of the vehicle's kinetic energy into elastic strain energy and dissipates energy through damping, thereby reducing the load transmitted to the main body and demonstrating the energy absorption capability of the anti-collision structure.

The von Mises equivalent stress distribution indicates that the maximum stress is mainly concentrated in the front region of the bumper and at its connection to the vehicle body, while the stress level in the body structure remains relatively low and uniformly distributed. Since no plastic constitutive behavior was introduced for the ABS material, the equivalent plastic strain remains zero throughout the simulation, suggesting that the structural response stays within the elastic range under the selected collision conditions.

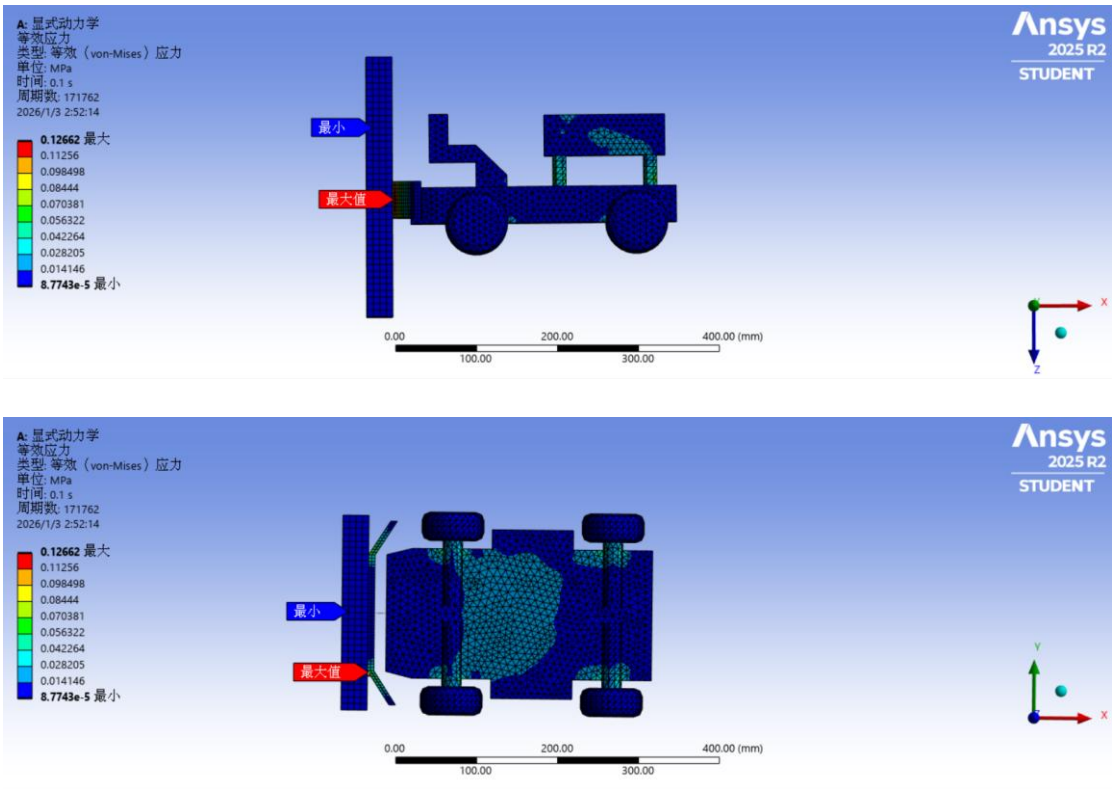


Figure 9 von Mises equivalent stress contours of the structure under Condition X (top: side view; bottom: bottom view)

In addition, the directional acceleration results indicate that the peak acceleration during collision reaches the order of $10^5 mm/s^2$, reflecting the strong transient impact characteristics of frontal collision. Owing to the presence of the bumper and the equivalent spring–damper system, the contact duration is effectively extended, which helps to limit the instantaneous acceleration peak transmitted to the vehicle body. From a dynamic response perspective, this behavior further demonstrates the buffering and impact mitigation effect of the anti-collision structure (Figure 10).

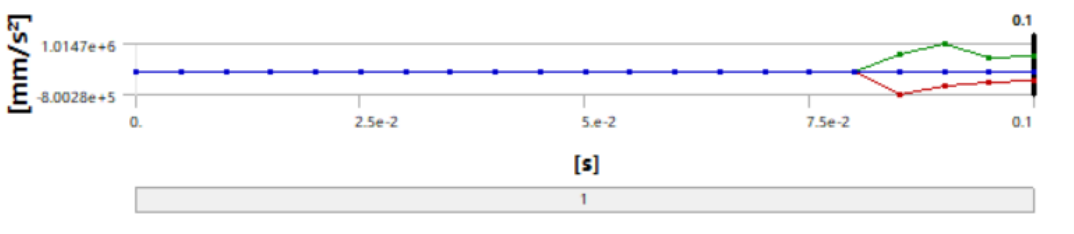


Figure 10 Time History Curve of X-Direction Acceleration

Overall, under the premise of reasonable modeling simplification and inertia equivalence, the explicit dynamic analysis based on the ABS material model can effectively capture the stress distribution characteristics, deformation modes, and energy absorption behavior of the vehicle during frontal collision. The results provide qualitative and semi-quantitative support for the effectiveness of the anti-collision structure and serve as a dynamic basis for the subsequent static stress analysis of the chassis.

3.2. Static Stress Analysis (SolidWorks Simulation)

The geometric model was developed based on the actual assembly configuration. The chassis consists of acrylic plates as the main structure and annealed 304 stainless steel L-brackets as key load-bearing components. For this static analysis, materials were assumed to be isotropic and linearly elastic. Material nonlinearity and large deformation effects were neglected to focus on the elastic response under rated loading conditions. This approach provides a consistent baseline for subsequent explicit dynamic analyses.

To simulate realistic mechanical constraints, bearing-type constraints were applied to the four motor shaft interfaces. This configuration restricts translational degrees of freedom while permitting rotation, accurately reflecting the axle–chassis interaction (Figure 11). A vertically downward distributed load of **900 N** was applied to the top plate to simulate the combined weight of onboard electronics and payload. The von Mises criterion was adopted to evaluate the structural strength.

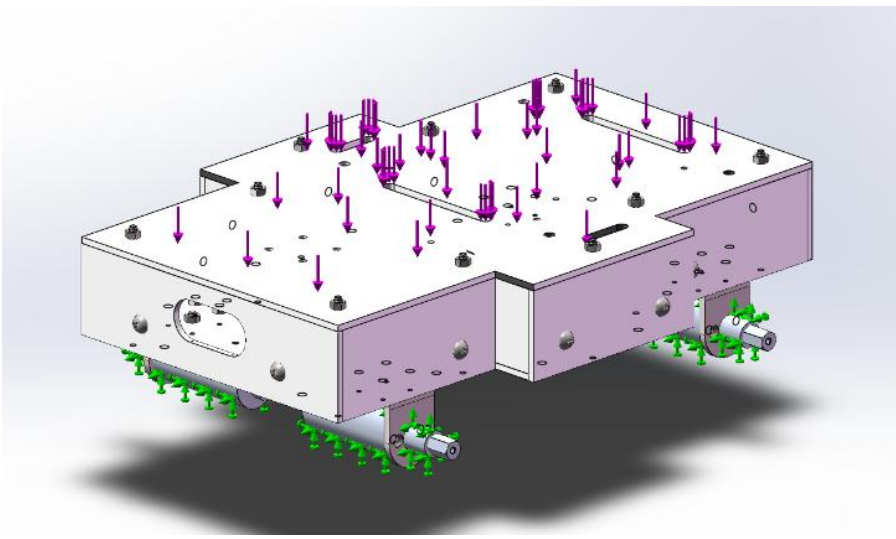


Figure 11 Schematic of loads and boundary conditions (SolidWorks output)

The Resultant Displacement (URES) contour shows that deformation is primarily concentrated in the loaded region of the top plate, with minimal displacement near the motor supports. The overall deformation is smooth and continuous, indicating a reasonable stiffness distribution (Figure 12).

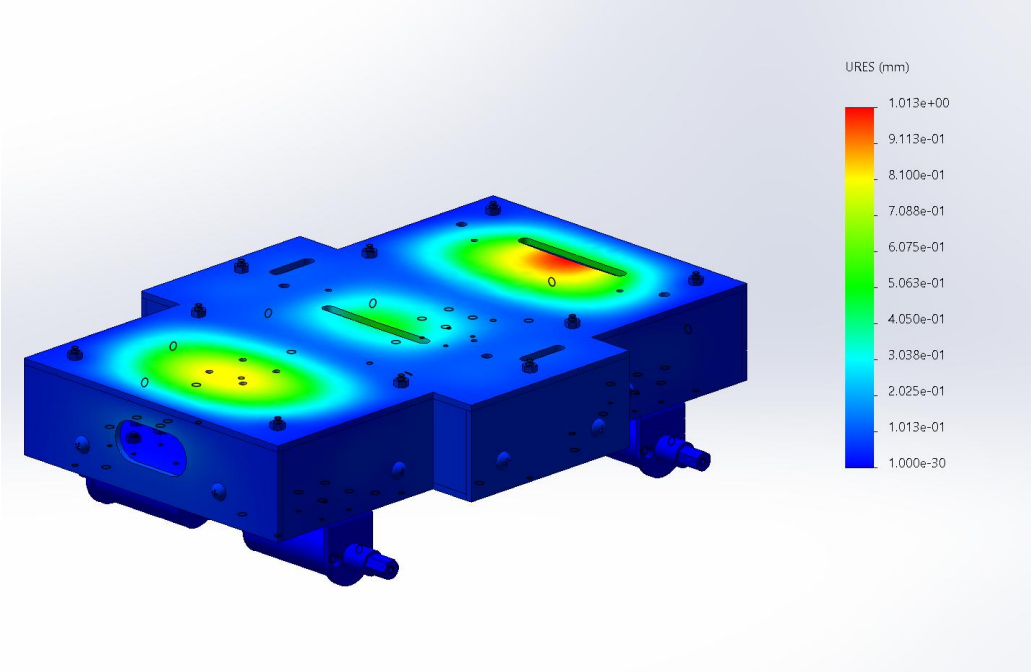


Figure 12 Resultant displacement (URES) contour of the chassis (bottom view, unit: mm)

The von Mises stress results indicate that the maximum stress occurs at the bottom of the right-rear L-bracket (Figure 13 and Figure 14). This location represents a geometric discontinuity along the primary load path, resulting in a typical stress concentration consistent with structural mechanics principles. Simulation results confirm that stresses in the acrylic plates are significantly below the material's allowable limit. For the annealed 304 stainless steel L-brackets (Yield Strength $\sigma_y \approx 205$ MPa), the calculated minimum **Safety Factor is approximately 1.69**. This confirms that the chassis structure satisfies static strength requirements under the rated operating load.

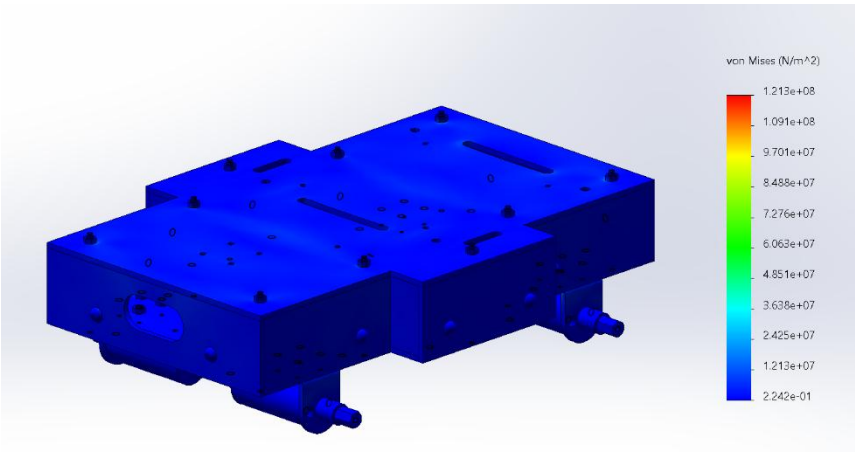


Figure 13 Overall von Mises equivalent stress contour of the chassis

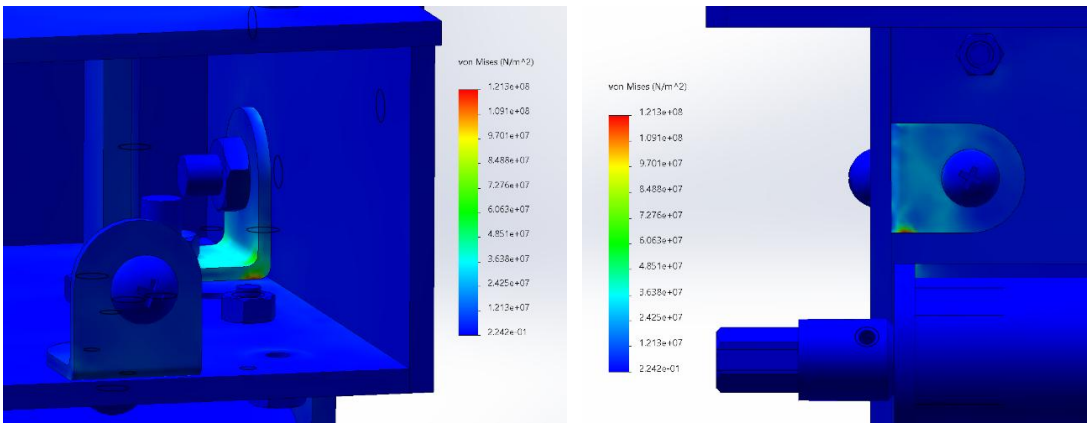


Figure 14 Local magnified view of stress concentration at the bottom of the L-bracket

3.3. Static–Dynamic Analysis Comparison and Engineering Significance

By combining explicit dynamic analysis and static stress analysis, the mechanical performance of the chassis structure is evaluated under both transient impact and normal operating conditions. The results show good consistency in identifying critical stress regions. In both analyses, the bumper–body connection and the bottom region of the L-bracket are identified as primary load transfer paths and potential stress concentration areas.

Under frontal collision conditions, the explicit dynamic analysis demonstrates that the bumper absorbs part of the impact energy through elastic deformation, reducing the stress and acceleration transmitted to the chassis body. The subsequent static analysis confirms that, under rated loads, the stress levels of the chassis and key load-bearing components remain within allowable material limits, with sufficient safety margin.

These results indicate that the chassis structure exhibits reasonable stiffness and strength under normal operation, while the bumper effectively provides impact buffering under collision conditions. The consistency between dynamic and static analyses verifies the rationality of the overall structural design.

3.4. Conclusions and Suggestions for Improvement

In this work, finite element simulations were carried out to study the structural response of the car chassis under static loading and frontal collision conditions. By combining static analysis with explicit dynamic analysis, the stress distribution, deformation characteristics, and impact buffering behavior of the chassis were evaluated.

The static analysis results show that, under the rated load, the chassis exhibits relatively small overall deformation. The deformation is mainly distributed around the loaded region and varies smoothly across the structure, without obvious local stiffness discontinuities. Meanwhile, the equivalent stresses in the main load-bearing components remain well below the corresponding material strength limits, indicating that the chassis meets the static strength requirements for normal operating conditions.

A comparison between static and dynamic results reveals that both analyses identify similar critical stress locations. In particular, the connection region between the bumper and the chassis (or the bottom of the L-bracket) consistently appears as a primary load transfer area with higher stress levels. This agreement suggests that the static analysis provides a reasonable preliminary indication of structurally sensitive regions, while the dynamic analysis further captures their transient behavior during impact.

The explicit dynamic analysis demonstrates that, during frontal collision, the front bumper is the first component to engage with the obstacle. Through elastic deformation, the bumper absorbs part of the kinetic energy and delays load transmission to the chassis body, resulting in reduced stress and acceleration responses in the main structure. Under the low-speed collision conditions considered in this study, the structural response remains within the elastic range, and no material yielding is observed, indicating an acceptable safety margin.

Overall, the combined static–dynamic finite element approach proves effective for evaluating both the strength and impact resistance of the chassis structure. The results provide a practical reference for further structural refinement and offer guidance for future engineering design and optimization.

4. Microcontroller Implementation

1. System Architecture and Core Control Unit

The system adopts a dual-core distributed control architecture to balance real-time performance requirements with computational power [6]. The primary motion controller utilizes an LPC1768 development board (Figure 15) based on the Mbed platform [7]. Leveraging the real-time characteristics of Mbed OS, it is responsible for chassis motion kinematics, sensor data acquisition, and low-level logic control. The vision processing unit employs a Raspberry Pi Zero 2 W (Figure 15) running a Linux system, which is dedicated to CSI camera image acquisition, processing, and gimbal servo control [8]. The two units exchange data bidirectionally via a UART serial interface (connecting p27/p28 of the LPC1768 to GPIO14/15 of the Raspberry Pi), achieving an organic integration of high-level visual decision-making and bottom-level motion execution.

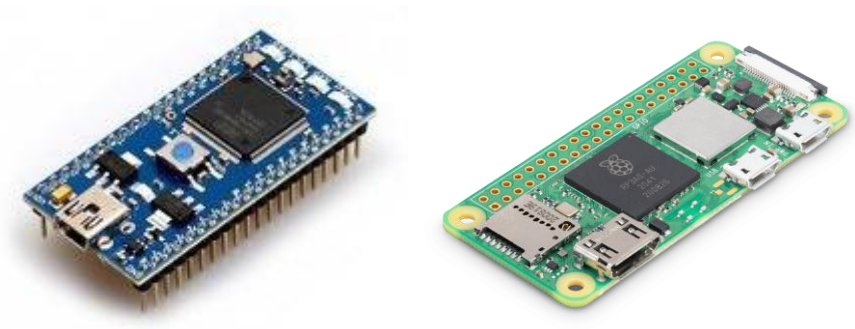


Figure 15 Mbed LPC1768 and Raspberry Pi Zero 2 W.

2. 4WD Chassis Drive System

At the actuation level, the system establishes a four-wheel independent drive (4WD) platform [9]. The LPC1768 drives four JGA25-370 DC geared motors via two L298N dual-channel full-bridge driver modules (Figure 16) [10]. Regarding control logic, the LPC1768 allocates 8 GPIO ports (p12–p19) connected to the IN1–IN4 pins of the two driver boards to control the rotational direction logic of the four motors. Simultaneously, four PWM output pins (p21–p24) are utilized to independently control the rotation speed of each wheel.

Figure 16 JGA25-370 DC geared motors and L298N dual-channel full-bridge driver module.

3. Vision Gimbal and Image Acquisition Processing System

The vision system operates independently within the Raspberry Pi subsystem. The Raspberry Pi Zero 2 W connects to a high-definition camera via the CSI interface for real-time image capture. To enable flexible viewing angle adjustments, GPIO12 and GPIO13 of the Raspberry Pi are connected to two MG90S metal gear servos (Figure 17), constructing a two-degree-of-freedom (2-DOF) pan-tilt gimbal. The Raspberry Pi generates two software PWM signals to control servo rotation, thereby driving the camera to perform horizontal and vertical scanning.

Figure 17 MG90S metal gear servo.

4. Ultrasonic Perception and Emergency Braking

To ensure navigational safety, the LPC1768 integrates an HC-SR04 ultrasonic ranging module (Figure 18) [11]. The system sends a periodic 10µs high-level pulse via pin p30 to the module's Trig terminal to trigger detection. Upon emitting ultrasonic waves and receiving the echo, the module outputs a high-level signal to pin p29 of the LPC1768 via the Echo terminal. The controller utilizes an internal timer to capture the duration of the high-level signal at p29 to precisely calculate the distance to obstacles. When the detected distance

falls below a safety threshold, the LPC1768 prioritizes interrupting current motion commands to force the chassis to stop, implementing active obstacle avoidance. Considering that the vehicle's inertia varies significantly with driving speed, the required physical braking distance also differs. Therefore, the system adopts a speed-dependent dynamic braking strategy—the higher the vehicle speed, the longer the set safety braking distance [12].

Figure 18 HC-SR04 ultrasonic ranging module.

5. Double-level Communication Network

The system establishes two layers of communication links (internal and external). The external link employs an HC-08 Bluetooth module (Figure 19), with its TX/RX pins cross-connected to ports p9/p10 of the LPC1768 [13]. This establishes a transparent wireless serial channel used for receiving remote control commands from mobile terminals. The internal link is located between the LPC1768 and the Raspberry Pi (p27/28 and GPIO14/15), used for transmitting visual recognition results or collaborative gimbal commands.

Figure 19 HC-08 bluetooth module.

6. Power Management and Distribution

The system adopts a hierarchical power supply strategy. A 12V lithium battery pack directly powers the two L298N driver modules to meet the high-power demands of the motors. Simultaneously, an LM2596 DC-DC buck converter module (Figure 20) is used to step down the 12V input to a stable 5V voltage, which is connected to the VIN terminals of the LPC1768 and HC-08 for their power supply [14]. The Raspberry Pi and servo section utilize an independent 5V power input.

Figure 20 LM2596 DC-DC buck converter module.

5. Electrical Circuit and PCB Design

The core of the project is a Bluetooth-controlled robotic vehicle featuring a dual-controller architecture designed to decouple computational tasks. This approach distributes the workload: the LPC1768 handles low-level chassis locomotion, while a Raspberry Pi Zero 2 W manages the upper mechanisms, servo actuation, and computer vision tasks. Based on the requirement in microcontroller implementation, designed the Hardware Connection Electrical Circuit shown below (Figure 21).

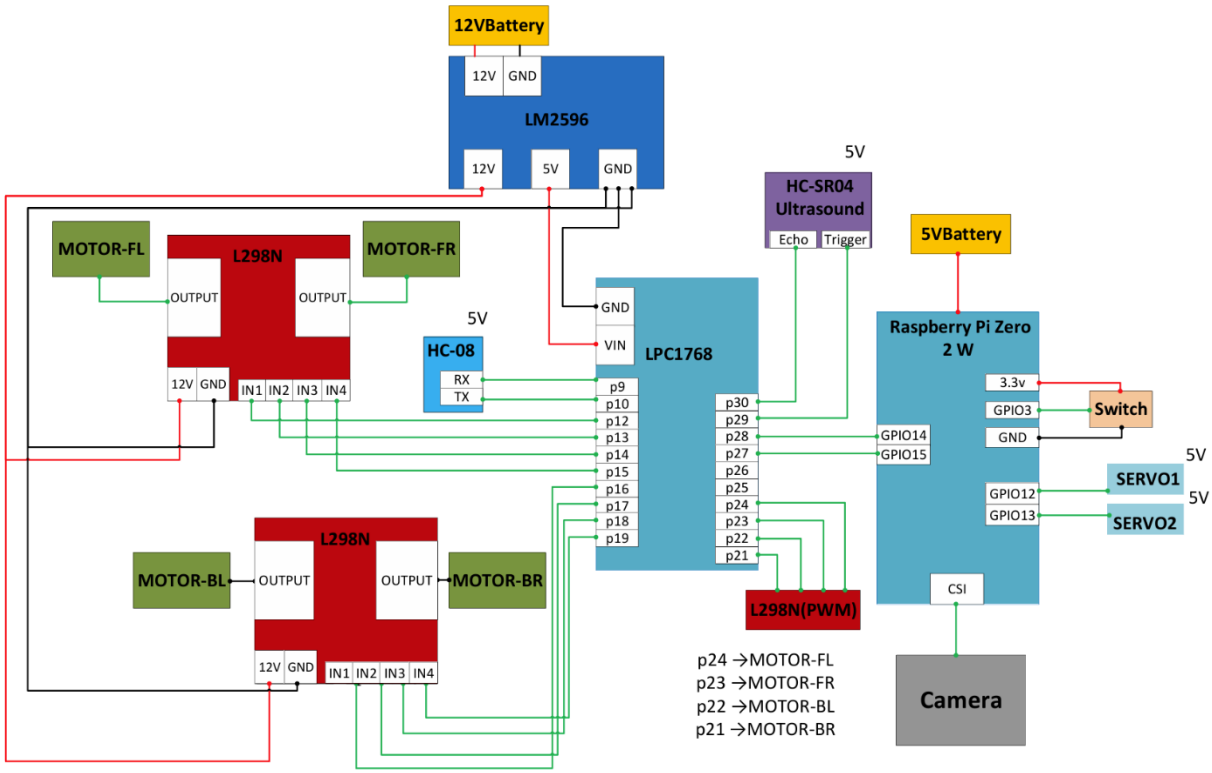


Figure 21 Hardware connection diagram.

5.1. LPC1768 Motion Control Subsystem

The LPC1768 microcontroller acts as the primary driver for the vehicle's chassis. It interfaces with a mobile

device via an HC-08 Bluetooth module (connected to pins p9/TX and p10/RX) to receive user commands. For propulsion, an H-Bridge driver circuit is employed to control four DC motors. Speed regulation is achieved through Pulse Width Modulation (PWM) on pins p21 through p24, while the logic direction control is mapped to DigitalOut pins p12 through p19 (Figure 22). Inter-controller communication is established via UART; the LPC1768 connects to the Raspberry Pi by cross-linking TX (p28) and RX (p27), supporting full-duplex data transfer at matching baud rates. Additionally, an ultrasonic sensor is integrated (p25, p26) to provide real-time distance data for obstacle avoidance.

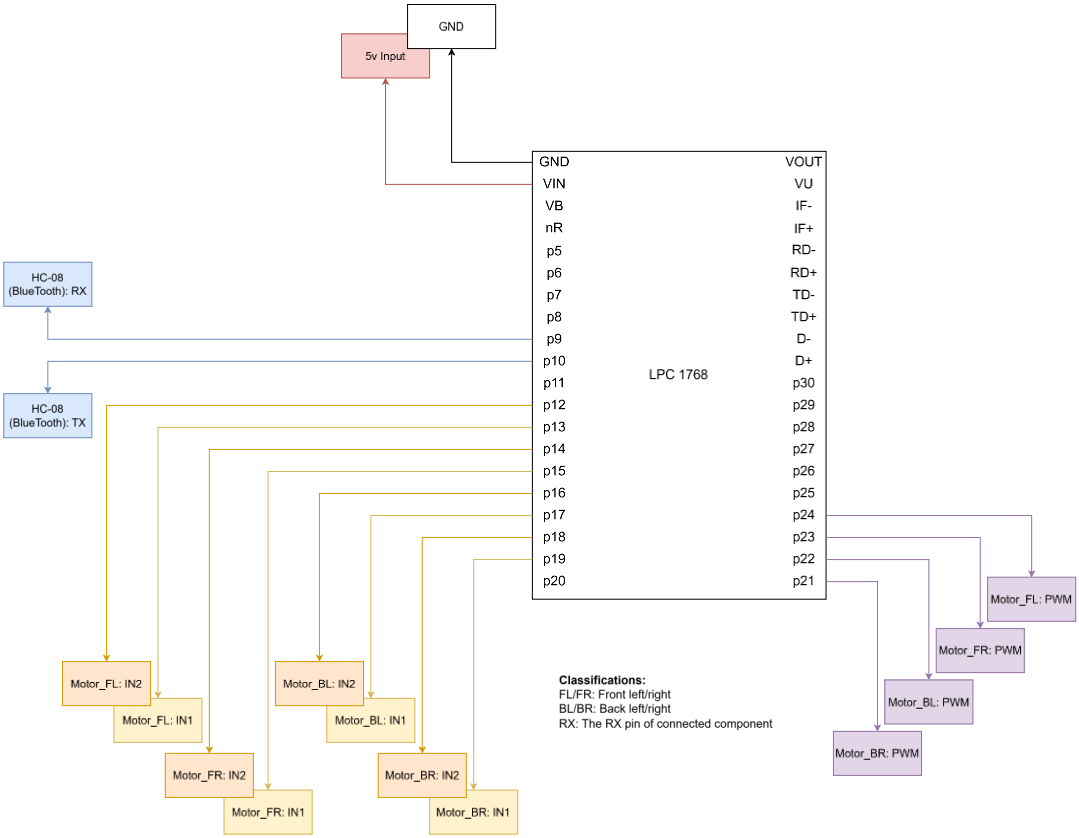


Figure 22 Wiring diagram of the LPC1768 subsystem showing connections for Motors, Bluetooth, and Power distribution.

5.2. Raspberry Pi Implementation and Design Iteration

A significant design change was implemented regarding the servo motors. Initially, we intended to control the servos using the LPC1768. However, during testing, we discovered a conflict with the PWM interfaces. The LPC1768's PWM pins share a common period; the DC motors required a different frequency than the strict 50Hz (20ms) period required by the servos. Adjusting the period for one affected the other, leading to unstable operation.

To resolve this, the servo control was moved to the Raspberry Pi Zero 2 W. The Pi drives the servos using hardware PWM on GPIO 12 and GPIO 13 (Figure 23). Beyond simple actuation, the Raspberry Pi is critical for data processing. It connects to a camera module, processing incoming video data to perform visual tasks

before sending logic signals or coordinating with the LPC1768 via UART (GPIO 14/15) (Figure 23).

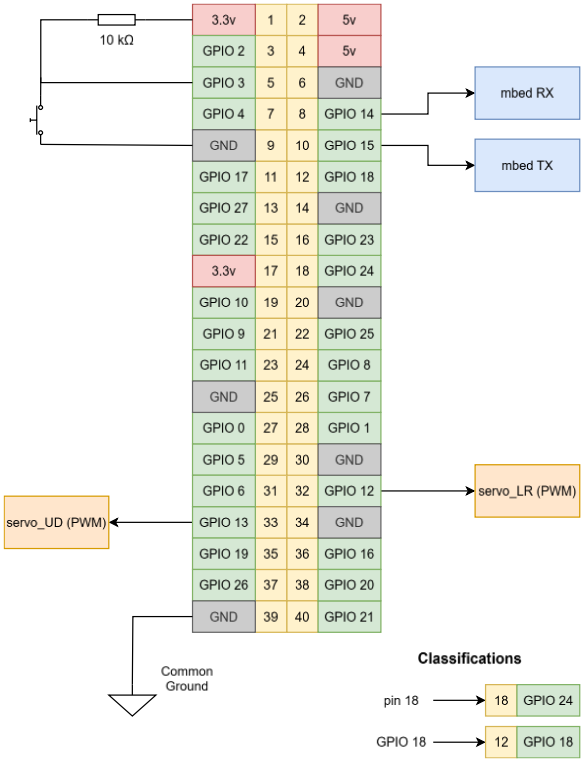


Figure 23 GPIO connection diagram for the Raspberry Pi Zero 2 W, detailing Servo PWM and UART connections.

5.3. PCB Design

To ensure reliability and reduce the mess of jumper wires, we designed custom PCBs for both subsystems. The schematics (Figure 24 and Figure 25) highlight key design features.

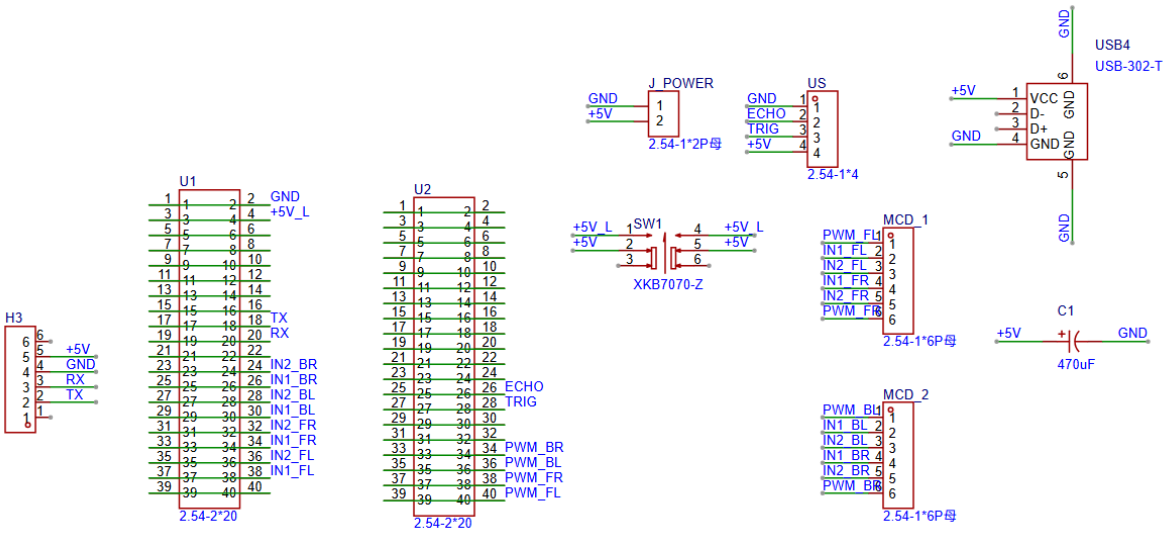


Figure 24 Circuit schematic for the LPC1768 carrier board, including motor driver interfaces and ultrasonic sensor

headers.

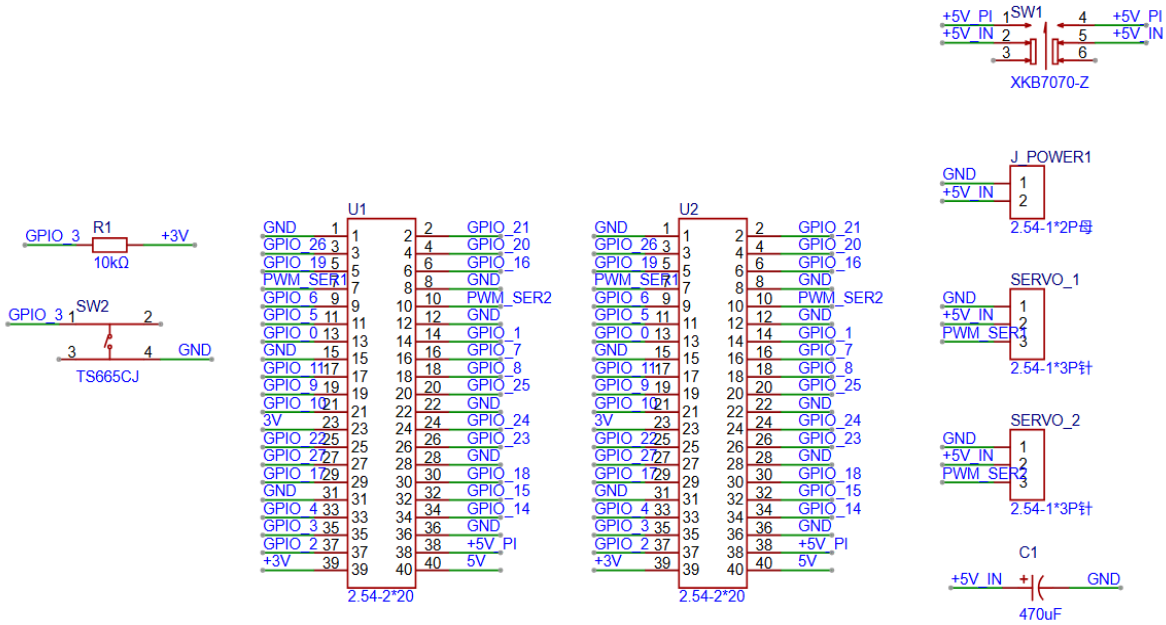


Figure 25 Circuit schematic for the Raspberry Pi interface board with protection components and switch input.

Power Filtering

We placed 470µF capacitors (C1) across the power rails. This was crucial for filtering out voltage drops caused by the sudden current draw of the motors and servos, protecting the logic levels of the microcontrollers.

Modular Interface

The boards use standard headers for the Mbed and Pi, allowing us to easily swap out the expensive microcontrollers if a fault occurs.

User Input

The Pi carrier board includes a user button (with a 10kΩ pull-up resistor) on GPIO 3 to trigger specific program modes or shut down the system safely.

5.4. Physical PCB layouts

Based on the circuit schematics, the physical PCB layouts were designed using EasyEDA. Special attention was paid to the routing width; power traces connected to the motor drivers and servos were widened to 40 mils to minimize resistance and heat. The finalized software layout designs for both boards are illustrated in Figure 26 and Figure 27.

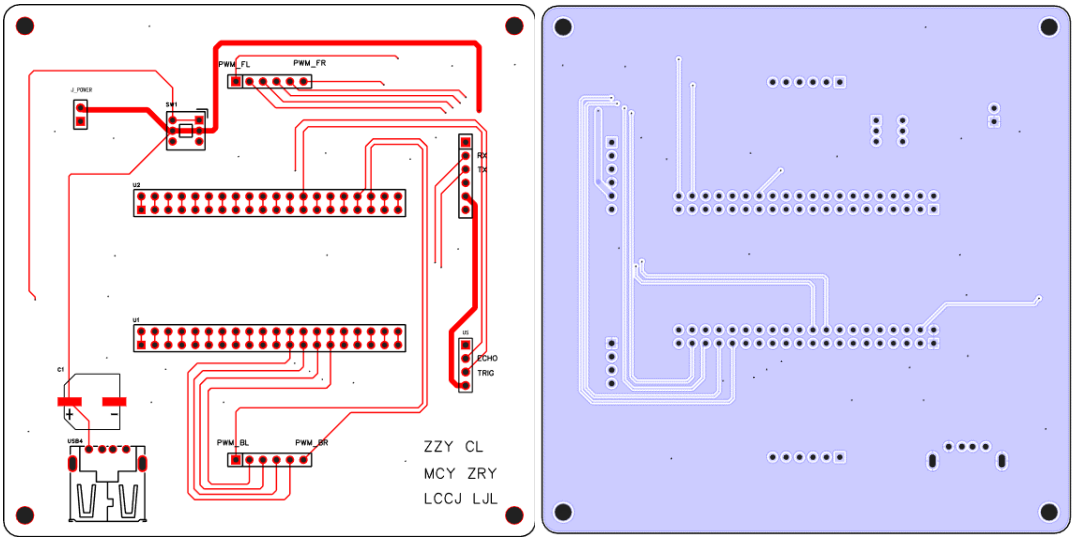


Figure 26 PCB layout design for the LPC1768 subsystem. The layout features wide traces for motor power lines to handle higher current.

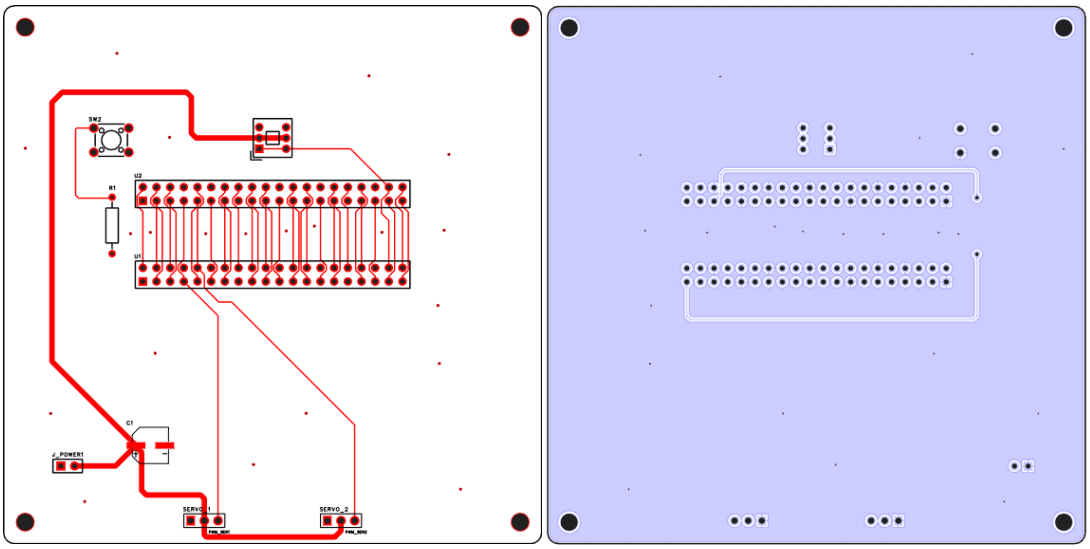


Figure 27 PCB layout view for the Raspberry Pi interface board, showing component footprints and signal routing.

5.5. PCB Fabrication

Following the schematic capture and layout design, the boards were professionally fabricated and manually soldered. The completed modules allowed for a compact integration of all components. The final assembled PCBs for the LPC1768 chassis controller and the Raspberry Pi upper-mechanism controller are shown in Figure 28 and Figure 29, respectively.

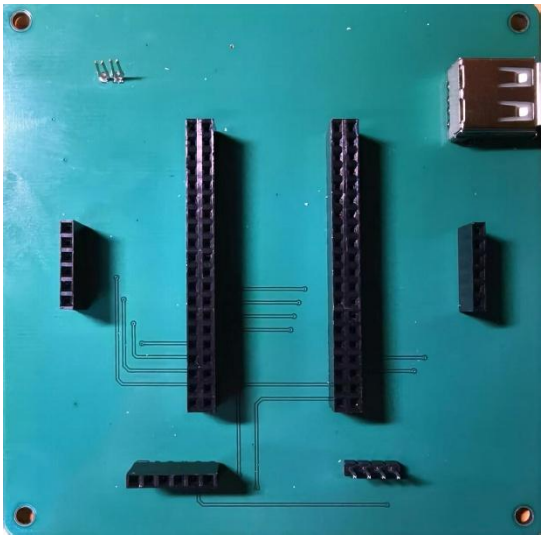


Figure 28 Fabricated LPC1768 Motion Control PCB populated with motor drivers headers and Bluetooth interface.

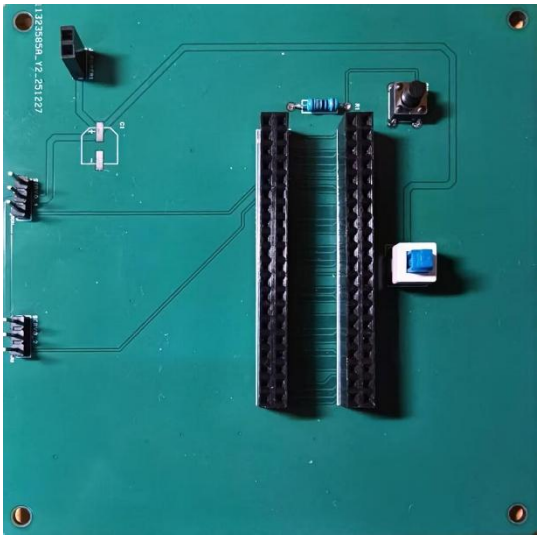


Figure 29 Assembled Raspberry Pi Interface PCB featuring servo headers, power stabilization, and user input switch.

Figure 30 demonstrates the circuit connection after assembly.

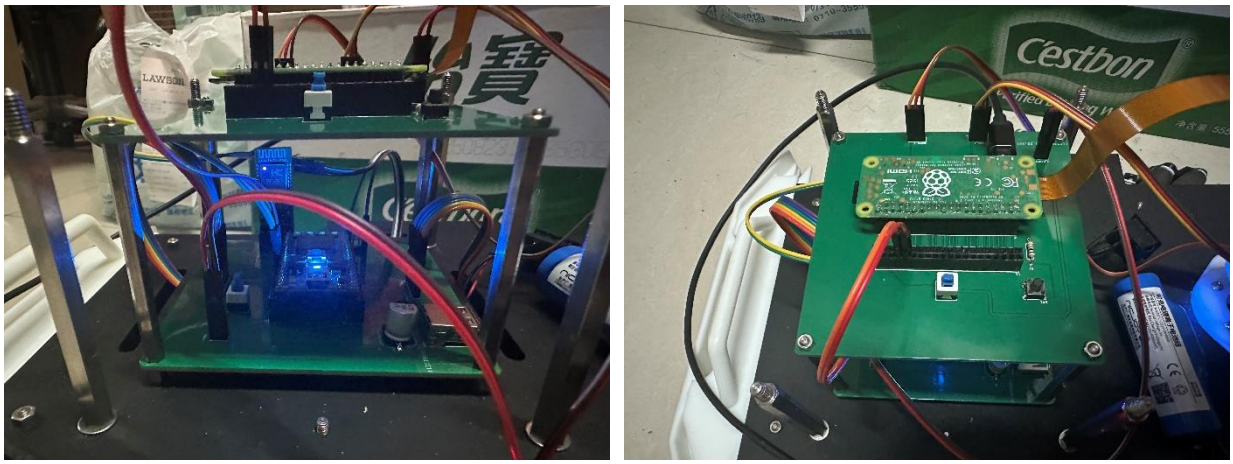


Figure 30 Connection of circuits (a) Lower board for LPC1768 (b) Upper board for Raspberry Pi.

6. Software Development

This section aims to give a detailed discussion on the logic of control programs.

In this project, the motion control of the car is implemented using LPC1768. However, the flash memory of the chip on LPC1768 is only 512 KB, which is far from sufficient to realize the visual function. Therefore, the vision module is split from LPC1768 and independently works Raspberry Pi Zero 2 W (referred to as Raspberry Pi in the following text).

6.1. Motion Control: LPC1768

6.1.1. Motion Control: Omnidirectional Movement Based on Mecanum Wheel

Control of a Single Motor

The control of a single motor is encapsuled into a class `StdMotor`. The `setSpeed` function accepts inputs from -100 to 100. It automatically clamps the value and toggles direction logic based on the sign. The speed percentage input is then converted into precise pulse width (in microseconds) based on a configurable period. Combined with ultrasound module, a bool variable `forwardLock` is set to avoid a forward collision. It will be triggered when the car is too close to a barrier. The threshold distance D, when the car should stop, is given by a linear function in Equation(2), where L_v is the speed level (no unit).

$$D = 7.5L_v + 5, 1 \leq L_v \leq 10 \tag{2}$$

This module simplifies motor driver management, allowing developers to focus on high-level motion algorithms.

Velocity Conversion for Mecanum Wheels

Assuming a rectangular robot chassis equipped with four Mecanum wheels (typically in an "X" configuration). The definitions of variables are listed in tab. The directions of variables are based on a right-handed coordinate system, as shown in *Figure 31*.

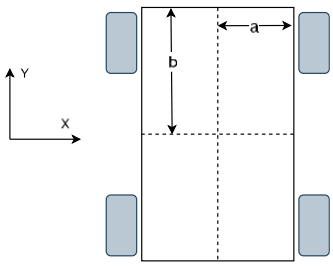


Figure 31 Size parameters of the car and the direction of the coordinate system (+y-axis indicates forward).

Variable	Definition

$v_{fl}, v_{fr}, v_{bl}, v_{br}$	Linear velocities of the Front-Left, Front-Right, Back-Left, and Back-Right wheels, respectively.
v_y	Longitudinal velocity of the car (Forward).
v_x	Lateral velocity of the car (Strafing Right).
ω_z	Angular velocity of the robot (Rotating Counter-Clockwise).
a	Half the distance between the left and right wheels (half-width).
b	Half the distance between the front and rear wheels (half-length).

Table 1 Variable definitions of the kinetic analysis.

When given the required speed in x and y axis, the speed of each wheel can be inversely calculated according to equation(3)(2).

$$\begin{cases} v_{fl} = v_y + v_x - (a+b)\omega_z \\ v_{fr} = v_y - v_x + (a+b)\omega_z \\ v_{bl} = v_y - v_x - (a+b)\omega_z \\ v_{br} = v_y + v_x + (a+b)\omega_z \end{cases} \tag{3}$$

In this project, the calculation of the rotating speed is simplified by only applying base speed v_B rather than $(a+b)\omega_Z$. The transformation between users' inputs and the final speed of each wheel will later be discussed in detail.

Conversion from Users' Inputs to Wheels' Speed

Users can choose to control the car in 'Buttons' mode or 'Joystick' mode, as shown in figure. In 'Buttons' mode, the car's movement is controlled by pressing and holding the directional keys (U, D, L, and R). When the key is touched, the phone will send an uppercase letter, indicating the beginning of movement in this direction. When released, a lowercase letter will be sent, indicating the end of the movement. The same rule applies to rotation as well (B and E). In 'Joystick' mode, the velocity is sent as a vector according to the position of the stick.

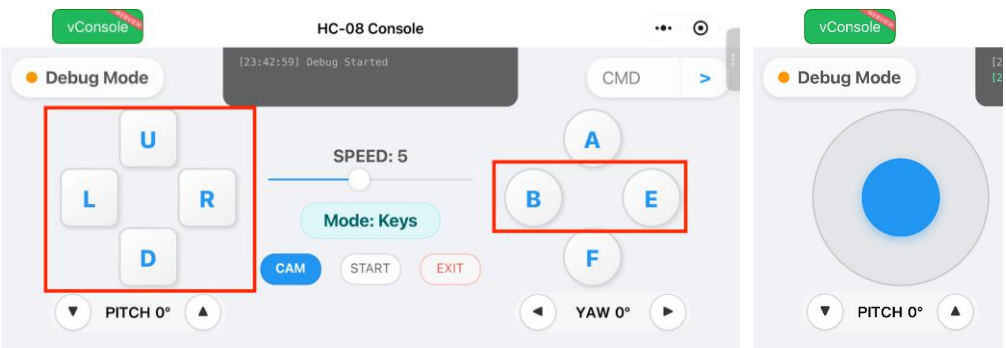


Figure 32 Control interface on WeChat mini-program. (a) 'Buttons (or Keys)' mode (b) 'Joystick (or stick)' mode.

Though the forms vary, inputs of these two modes will be normalized as a vector unit to indicate direction. The difference between these two modes processing of the information of the velocity. In 'Buttons' mode, the information of directions is first stored in a structure named `Direction` with four bool variables (L, R, U and D) recording which direction to go. It will then be transformed into a direction vecotor. Such a transition allows the movement in diagonal directions. The direction vector is then normalized to be a unit vector.

With the normalized direction vector, v_x and v_y can be easily calculated by multiplying a scalar - the base speed, which can be adjusted by dragging the slider in the middle. The speed of each wheel is then calculated using the equation(3).

All the functions related to motion control are encapsulated in the class `MotionControl`.

6.1.2. Message Sending and Receiving: Multithread Programming

The interaction between the user's phone and the microcontrollers is implemented by **Bluetooth communication protocol**. The Bluetooth module HC-08 is connected to LPC1768 as a **UART serial** device. `Mbed OS 6` offers a class, BufferedSerial, to manage serial devices in a safe and robust manner, supporting read/write operations, interrupt registration, and more.

Receiving Messages

For I/O-bound operations, such as network requests, database queries, or file access, multi-threading is essential for efficiency. Since I/O speeds are significantly slower than CPU processing, a single thread often leaves the CPU idle (blocked) while waiting for data. Multi-threading allows the CPU to switch to other productive tasks while one thread waits for an I/O response. This concurrency effectively hides latency and maximizes CPU utilization, significantly improving the program's overall throughput and responsiveness [15].

In embedded development, where the performance of the hardware is severely limited, the **multithreading idea** is even more crucial. The message receiver of LPC1768 is running on an independent thread, which is idle when there is no message. When a message arrives at the serial, it will trigger an interrupt bounded with the ports. In the interrupt, a flag will be set (Note that the flag is checked by bits, the 0th bit is for Bluetooth serial and the 1st bit is for Raspberry Pi). When recognizing the flag, the read thread begins to work, receiving characters and putting them into its buffer. When it meets the end of the message (or the buffer is full), it will pack the buffer and put it into the mailbox (`Mail` in `Mbed OS`). The flow chart in *Figure 33* helps demonstrate how the reading module works.

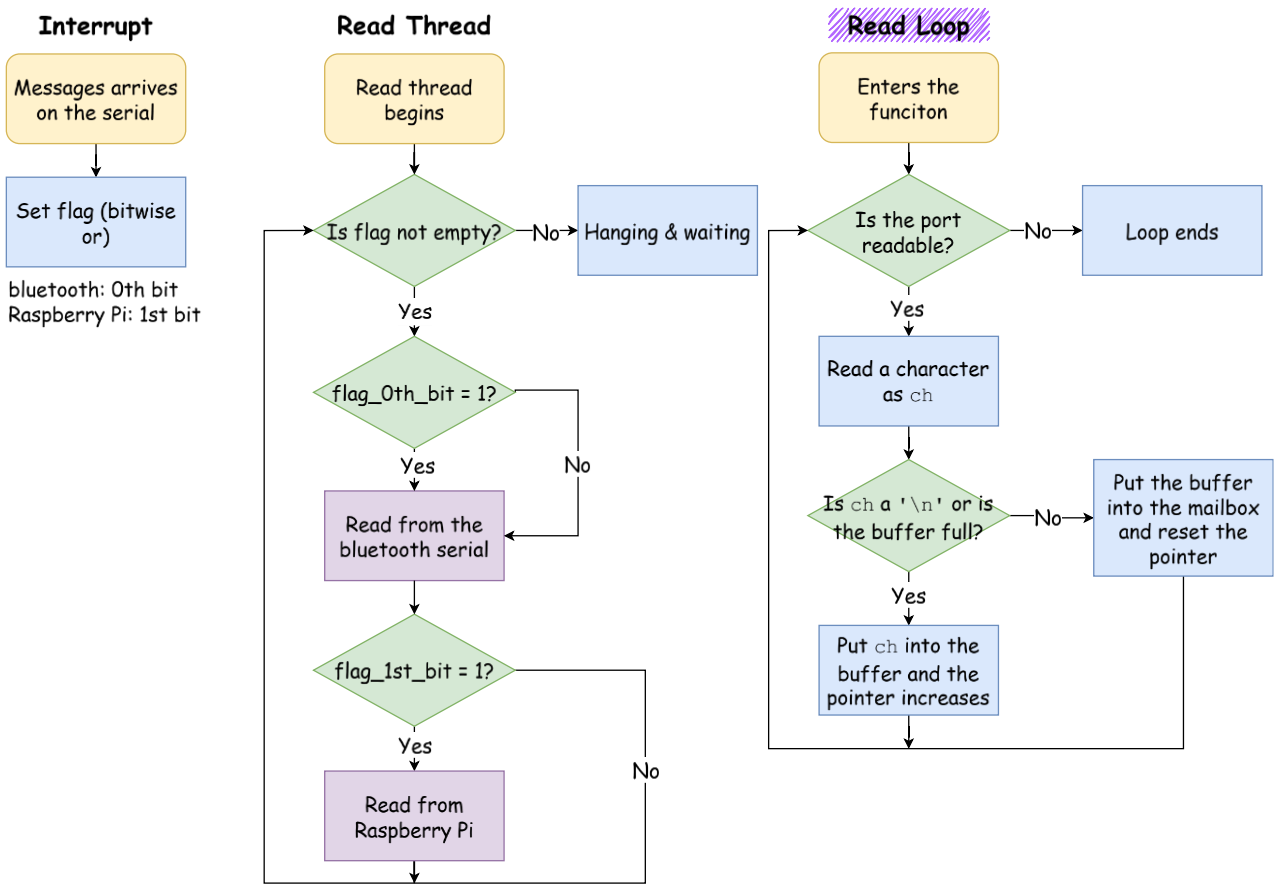


Figure 33 Flow chart of the reading module (including the serial port interrupt, read thread and the read loop).

The task of the main thread is simple, who is just responsible for taking out and parsing the messages in the mailbox. As *Figure 34* shows, the main thread will loop with a short waiting period when the mailbox is empty. In the normal loop, it will detect whether there is a potential collision and determine whether the motors should be locked (the `forwardLock` function). When there is a message in the mailbox, it will begin to analyze the message. With the thread scheduling, the program will not be blocked by waiting for messages.

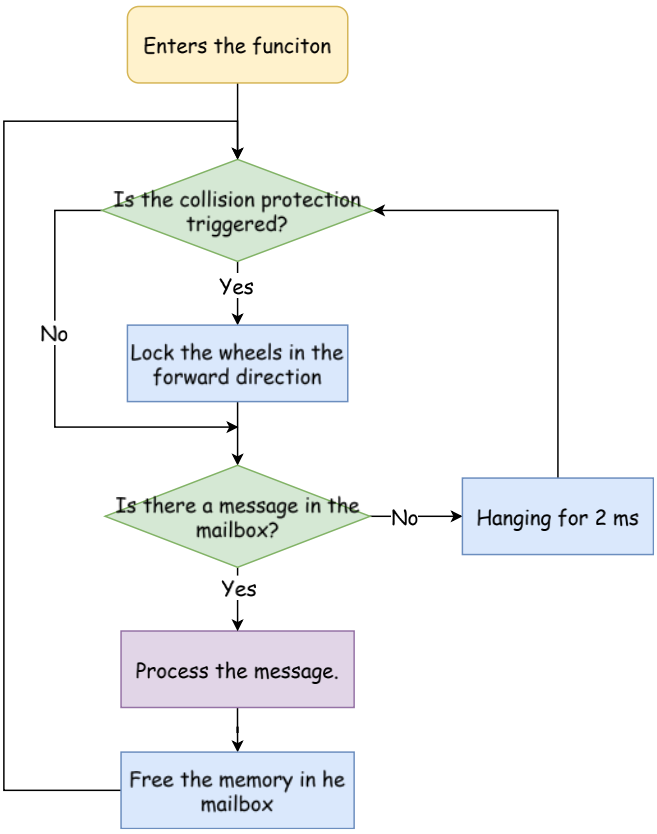


Figure 34 Flow chart of the main thread (LPC1768)

The `Mail` class in `mbed OS` is a robust Inter-Process Communication (IPC) mechanism that effectively combines the function of a **Queue** and a **Memory Pool**. Like the `queue` in the Standard Libraries of C++, `Mail` allows developers to exchange typed data structures (objects) directly between threads. It manages a fixed-size pool of memory blocks internally. The workflow involves the sender allocating a block (`alloc`), filling it with data, and posting it (`put`). The receiver then retrieves the data (`get`) and releases the memory (`free`) after processing. This design ensures thread safety and significantly reduces memory fragmentation by avoiding manual dynamic allocation.

Parsing the Commands

The commands can be categorized into two groups: single letter, beginning with a `#`; multi-parameter commands, beginning with a `:`.

The single-letter command corresponds to the buttons on the left and right pads. As mentioned above, the message will be sent when pressing or releasing the button (uppercase letters for the start and lowercase letters for the end). For instance, `# L` means starting to translate to the right and `# u` means stopping moving forward.

In multi-parameter commands, more information is included. For example, `: SVO UD 45` stands for turning the up-down directional servo to the position of 45 degrees.

The detailed description of the instruction specifications is attached to *Appendix A, Command Manual*. Here is just a brief introduction.

6.2. Vision: Raspberry Pi Zero 2 W

The Raspberry Pi operates on a complete Linux system (*Figure 35*), providing a robust toolchain for development. The development board is connected to the PC through SSH and Wi-Fi. The software is implemented in Python, which supports vast, powerful libraries, such as `gpiozero` for efficient GPIO management and `OpenCV` for machine vision tasks. With this foundational framework established, the following subsections will elaborate on the specific program logic and implementation details.

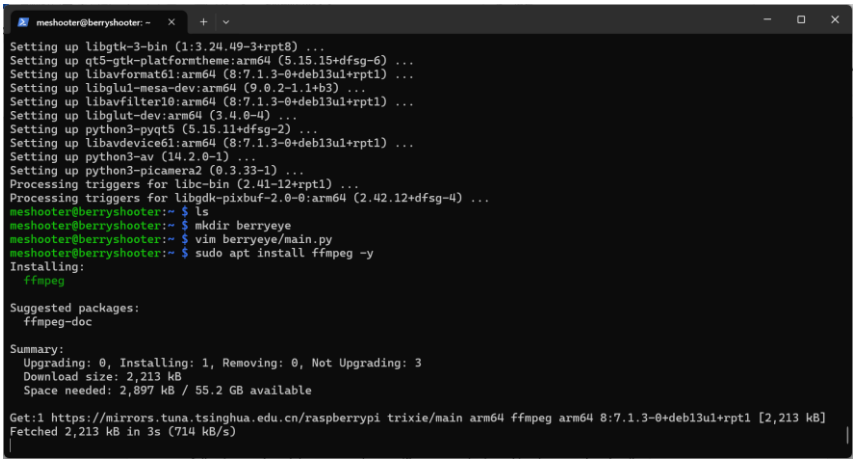


Figure 35 The development environment of the Raspberry. Due to the performance Raspberry Pi Zero 2 W, a lite version system without a desktop is installe.

6.2.1. Communication with LPC1768 and Gimbal Control

The communications between microcontrollers also use the **UART serial protocol**. The working principle of the serial module on Raspberry Pi is identical to the message processing methods on LPC1768: Create a thread for reading messages. Keep hanging until the content arrives. Send the instruction to the mailbox, which will be analyzed by the main program. The serial port is managed by the `serial` library, and these functions are packaged into `serial_module`.

The PWM interfaces in `gpiozero` adopt a software wave, which has strong distortions and fluctuations when controlling a servo. As a result, `pigpiod`, which offers a hardware PWM wave of high quality, is introduced to drive the servo. There is a linear mapping between the pulse width and the position of the servo (measured in degrees). The pulse-width ranges from 0.5ms to 2.5ms, with the position ranging from 0° to 180° degrees (-90° to 90° in practical use). Defined as an `AngularServo`, the servo can be easily controlled by calling a library function or assigning its member variables.

6.2.2. Machine Vision: Face Capture and Tracking

The `CameraStreamer` module serves as the core visual processing unit for the Raspberry Pi-based monitoring and tracking system. Built upon the `Picamera2` library and `OpenCV`, it is designed to handle

real-time video streaming, AI-powered object detection, and media recording. The module employs a robust multi-threaded architecture, separating video capture from heavy computational tasks to ensure a smooth frame rate for the end-user while performing image analysis in the background.

As illustrated in *Figure 36*, the module operates primarily on two concurrent threads: the **Camera Thread** and the **AI Thread**.

The Camera Thread (Main Loop)

This thread manages the hardware lifecycle and the video pipeline.

- **Dual-Stream Capture**: It utilizes `Picamera2` to configure two simultaneous streams: a high-resolution RGB stream ('Main') for user display and recording, and a low-resolution YUV stream ('Lores') optimized for AI processing.
- **Visualization**: It reads shared variables (`detected_faces` or `detected_bodies`) populated by the AI thread. If targets are detected, it draws bounding boxes (Red for faces, Green for bodies) directly onto the frame using `cv2.rectangle`.
- **Encoding & Broadcasting**: The processed RGB frame is for optional video recording (`VideoWriter`).. The image will then be compressed into JPEG/Base64 format to be broadcast via WebSockets to the client mini-program.
- **Resource Management**: To save power, the thread implements 'lazy loading', automatically releasing camera hardware when no clients are connected.

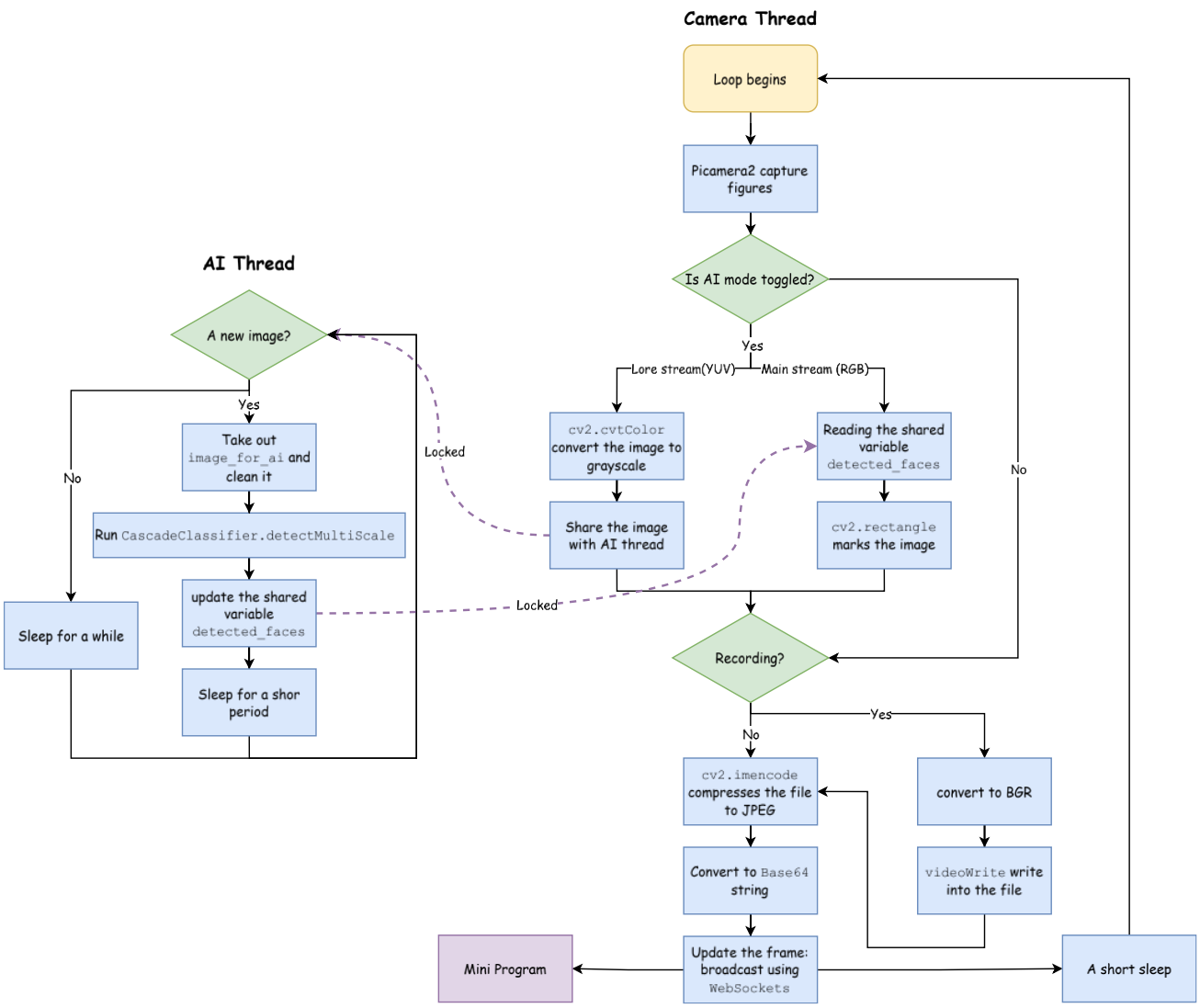


Figure 36 Flow chart of the visual module (AI thread and camera thread).

The AI Thread (Asynchronous Processing)

This thread runs effectively in the background.

- **Data Acquisition**: It continuously checks for new grayscale frames converted from the "Lores" stream.
- **Detection Logic**: Using **Haar Cascade classifiers**, it detects targets based on the current mode:

  Normal Mode: Detects faces for visual feedback (red boxes) but returns empty coordinates to the main controller to prevent servo movement.

  Tracking Mode: Detects full bodies (green boxes) and updates `primary_targets` to drive the servo motors.
- **Synchronization**: Thread locks (`threading.Lock`) are used extensively when updating shared coordinate variables to prevent race conditions.

The **Haar Cascade Classifier** is a robust machine learning algorithm used for real-time object detection, originally proposed by Viola and Jones. It functions by analyzing '**Haar-like features'**, which represent

intensity differences between rectangular image regions to identify patterns like faces or bodies. To optimize performance, the algorithm employs '**Integral Images**' for rapid computation and a '**Cascade**' structure to quickly filter out non-target areas. This multi-stage filtering process significantly reduces computational load, making it exceptionally efficient for embedded systems like the Raspberry Pi, where low latency and lightweight processing are essential for smooth performance. [16] *Figure 37* demonstrates the actual effects of the facial recognition.

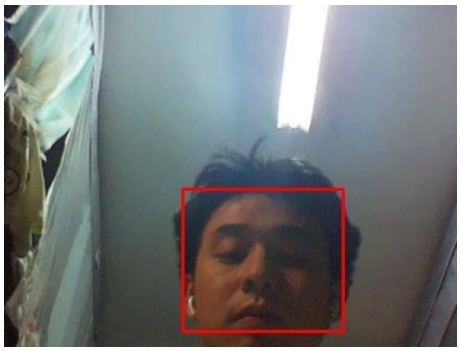


Figure 37 The actual effect of facial recognition, using Haar Cascade Classifier.

6.2.3. Main program: Asynchronous Thread Scheduling & Image Transmission

When `main.py` is started, it handles the initialization of GPIO servos and instantiates the `CameraStreamer` and `SerialManager` classes. The kernel of this script lies in its **hybrid concurrency model**, which simultaneously combines **Multi-threading** (for blocking hardware I/O) with **Python** `asyncio` (for high-concurrency network I/O).

The program is built upon the `asyncio` event loop. Unlike traditional threaded network servers, `asyncio` uses **cooperative multitasking**.[17] In the context of video streaming, this is crucial. The WebSocket server (`ws_handler`) needs to push large `Base64` video packets to the smartphone. In a synchronous blocking model, a slow network connection could block the entire CPU.

By using `async` and `await`, the program allows the WebSocket transmission to be 'pausable'. When the code executes the command `await websocket.send(data)`, if the network buffer is full, the function yields control back to the event loop. This allows the CPU to execute other tasks, such as processing UART callbacks or calculating AI coordinates, while waiting for the network to be ready, ensuring the system remains responsive even under heavy load. *Figure 38* drafts the working flow of the `ws_handler` thread in `main.py`. The reason of using WebSockets and Base64 will be discussed in the design of the Mini Program.

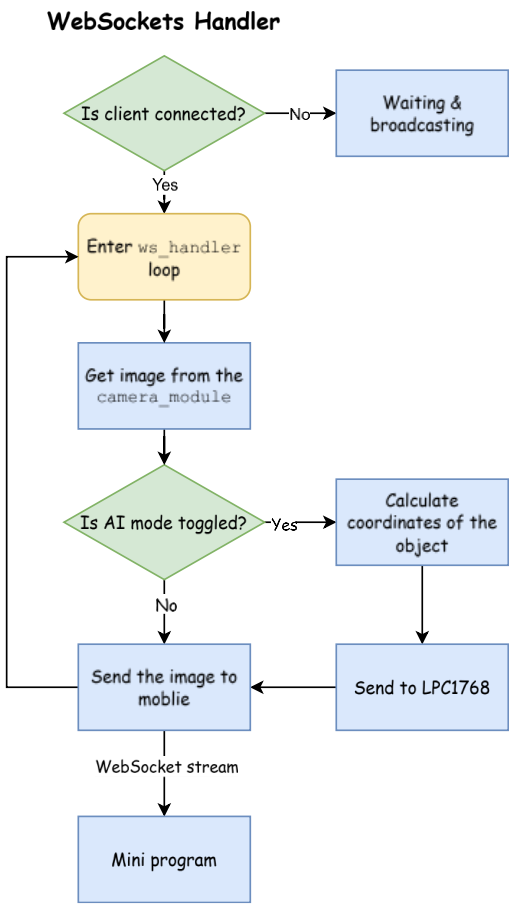


Figure 38 Flow chart of the WebSocket handler.

Once the background threads (Camera and UART) are launched, the main thread enters the `ws_handler` asynchronous loop, which orchestrates two critical workflows continuously:

- **Real-Time Video Telemetry (The Stream)**

  Inside the infinite `while True` loop, the program polls the `CameraStreamer` for the latest video frame. It compares frame IDs to ensure no duplicate data is sent. If a new frame is available, it is pushed to the client. The command `await asyncio.sleep(0.04)` is strategically placed here. It acts as a rate limiter, capping the stream at approximately 25 FPS to conserve bandwidth, while also forcing a "context switch" to ensure other pending tasks in the event loop get a chance to run.

- **Visual Function (The AI Feedback Loop)**

  This segment implements the logic for autonomous tracking. In every cycle, the program checks if the AI module is active. It accesses the shared memory `detected_faces` (protected by a thread lock) to retrieve the coordinates of the target and then sends them to the LPC1768 to execute a PID control.

6.2.4. Automatic Tracking with PID (Unsuccessful)

As one of the core functions of the visual car, the follow-up filming is supposed to be realized through a simple PID control. The position and size of the box indicating the human body is sent to the LPC1768 as an

input to the control loop. The PID loop mainly focuses on two aspects: the distance between the current position and the centroid of the canvas (left or right?); the area of the box (close or distant?). The PID formula can be given in a discrete form:

$$Output = K_p \cdot E + K_i \cdot \sum E + K_d \cdot (E - E_{last}) \tag{4}$$

In Equation(4), the output can be the turning parameters and the speed parameter.

However, in Haar Cascade Classifier, if a full human body is required to be recognized, the object should be exposed to the camera from head to feet (Figure 39), which is too strict. Since the car is near to the ground, it is hard to shot a photo involving a whole human body. Also, due to the poor anti-interference ability of Haar Cascade Classifier, the recognition results are often unstable, which is a strong disturbance of the PID control.

Hence, this function was removed in the final version of the project.

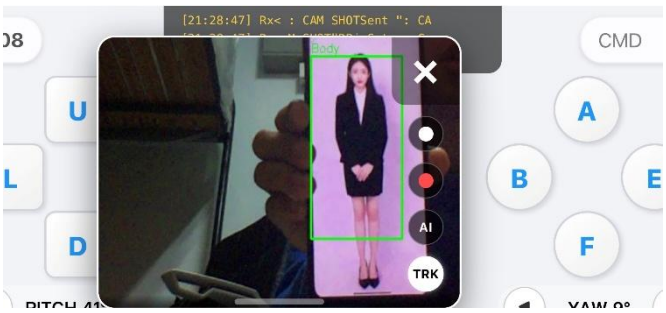


Figure 39 The actual effect of the full body recognition. It works only when the object's head and feet are included.

6.3. User Interaction: WeChat Mini Program

Most users are not equipped with the knowledge in embedded development and it is confusing to control the car on a console with only command lines. Therefore, bridging the gap between complex hardware logic and user experience is of great significance, which is also the core philosophy of this Mini Program. The application is designed to be more than just a remote controller; it is an intuitive dashboard that prioritizes responsiveness and feedback.

This Mini Program serves as a comprehensive control console for a Bluetooth and Wi-Fi based cars. It leverages WeChat's hardware API capabilities to establish a dual-link communication system: **Bluetooth Low Energy (BLE)** for low-latency command transmission and **WebSockets** for high-bandwidth video telemetry.

A. Dual-Mode Motion Control System

As introduced in the motion control subsection (*Figure 32*), the controller offers two distinct operational ways to suit different driving scenarios:

- **Discrete Button Mode:** Utilizes specific character commands (e.g., `# U` for forward, `# L` for left). This mode detects touch-start and touch-end events to trigger "start" and "stop" signals.

- **Analog Joystick Mode:** Implements a virtual joystick using vector mathematics. The code calculates the touch point's distance and angle relative to the center (`Math.atan2`), normalizing the output into a velocity vector `(Vx, Vy)`. It includes a frequency limiter (100ms interval) to prevent Bluetooth congestion and a "dead zone" algorithm to ensure smooth acceleration and deceleration.

B. Real-Time Visual Telemetry

According to the regulations of the WeChat platform, only enterprise users have access to the live-player API. As a result, conventional live stream protocols such as RTMP (Real-Time Messaging Protocol) and RTSP/RTP (Real-Time Streaming Protocol / Real-time Transport Protocol) are banned from the individual developers. In this project, as a substitution, **WebSockets** (using **Base64** encoding) combined with **Canvas 2D** is applied to realize the visual system in this project. The appearance is demonstrated in *Figure 40*

WebSockets is a communication protocol that provides full-duplex communication channels over a single TCP connection. Unlike standard HTTP requests, it establishes a persistent connection, allowing the server (microcontroller) to push data to the client (Mini Program) actively. This is essential for real-time applications like low-latency video streaming.[18] **Base64** is a binary-to-text encoding scheme that translates binary data—such as image frames—into an ASCII string format. In this project, Base64 ensures that raw camera data is compatible with the text-based WebSocket protocol, allowing video frames to be safely transmitted and reconstructed by the client for display.[19] **Canvas 2D** is a modern rendering context in WeChat Mini Programs (type="2d"). It aligns with the standard HTML5 API and leverages hardware acceleration. This ensures high-performance graphics processing, making it essential for smoothly rendering real-time video frames decoded from data streams.[20]

- **Stream Rendering:** The app connects to a video server (a Python script on Raspberry Pi) and receives MJPEG frames encoded in Base64 (in text formats). These are instantly decoded and drawn onto a hardware-accelerated Canvas node, ensuring low latency and avoiding the flickering of the screen.
- **Snapshot & Gallery:** The system listens for a specific protocol header (`PHOTO:`). Upon reception, it utilizes the File System Manager to write the Base64 data to a temporary local file and invokes the system's album API to save the image, which requires users to open settings. The program also supports video recording, which will be stored locally in Raspberry Pi.
- **AI Features:** Users can tap on the 'AI' or 'TRK' buttons to start a simple object recognition or start tracking the object though machine vision and PID control.

Figure 40 Video window of the controller, 'PITCH' and 'YAW' are used to control the gimbal.

C. Gimbal and Servo Control

The program also includes an easy access for controlling a 2-axis camera gimbal. The code manages the YAW (Left/Right) and PITCH (Up/Down) angles. It implements software limits (clamping angles between -90° and +90° or -60° and 90°) to prevent hardware damage. Moreover, it allows users to continuously change the angle by a long press.

7. Experiments and Results

To validate the functional performance of the Multifunctional Smart Border Patrol Vehicle, a comprehensive testing strategy was implemented. The testing phase was structured into three distinct tiers: component unit testing, subsystem integration testing, and a final simulated field operation. Figure 41 demonstrates the final work after assembly.

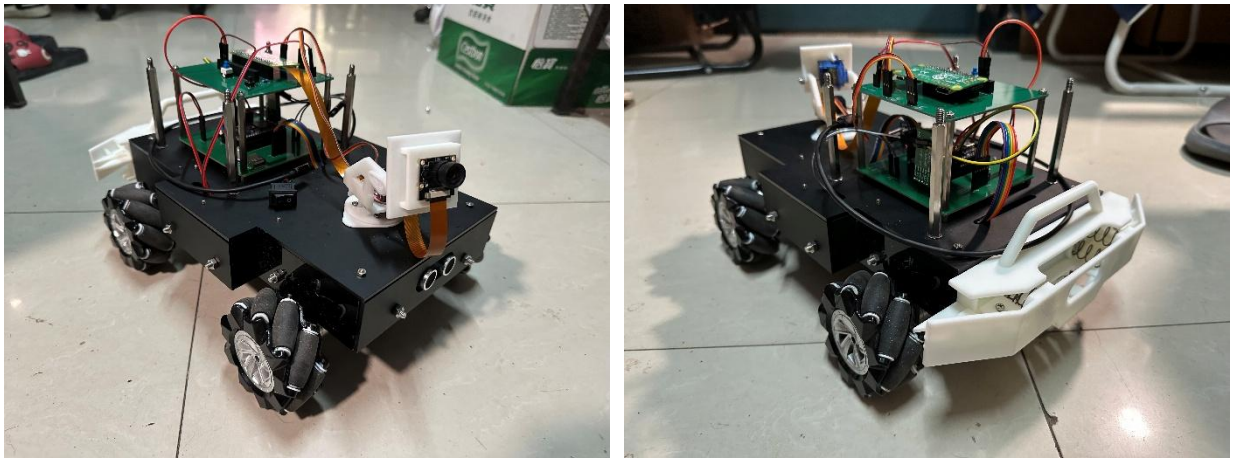


Figure 41 The complete car after assembly. (a) Front. (b) Back.

Locomotion and Kinematics Analysis

To verify the correctness of the kinematic model of the Mecanum wheel chassis and the LPC1768 motion control algorithm, we conducted a systematic motion test on the trolley on a flat indoor ground. The test content covers four main degrees of freedom: longitudinal travel, in-place rotation, lateral translation and

diagonal motion. During the experiment, the operator sent discrete instructions through the Bluetooth terminal to observe the actual trajectory response of the trolley (Figure 42). The results show that the control system can accurately parse the logical truth table and drive the L298N module to output the correct motor rotation combination.

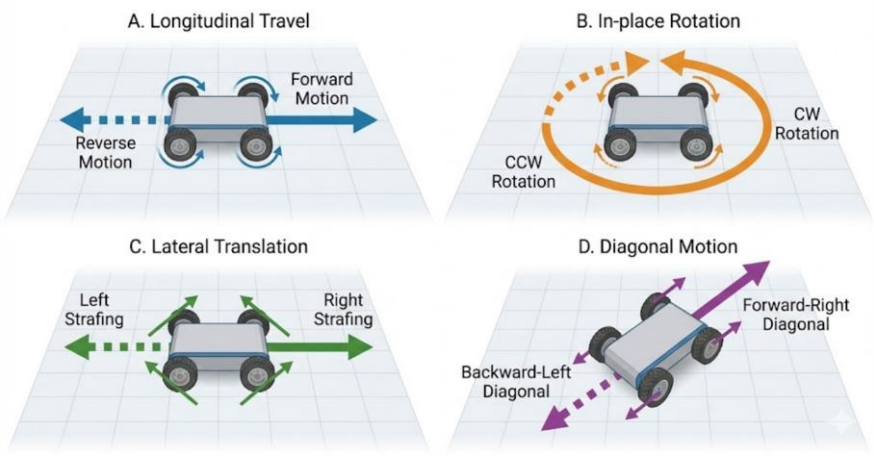


Figure 42 Systematic Kinematic Testing

Vision System and Remote Surveillance Performance

Following mechanical validation, the video transmission and face recognition systems were evaluated under varying environmental conditions. At a resolution of 480x320 and a frame rate of 30fps, the video stream latency was measured at approximately 100ms. In practice, this latency was imperceptible to the operator and was deemed fully adequate for real-time teleoperation.

The facial recognition algorithm operates in bright indoor lighting (approximately 300 lux) and dim corridor lighting (50 lux). Under bright conditions, the system achieved a detection accuracy of 95% within a range of 1.5 meters. It still performs well in dim light, as shown in the application interface in Figure 43. However, the accuracy rate has slightly declined, which suggests that we can add a lighting system to meet the needs of night operations.

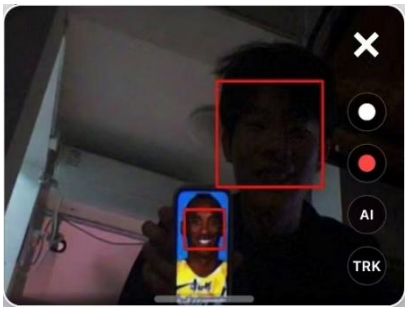


Figure 43 Recognize human faces in both bright and dim conditions

Communication Reliability and Control

To ensure the security of the platform during operation, tests were conducted on communication

reliability and autonomous navigation. In an open environment, the Bluetooth control link remains stable within 18 meters. When the distance exceeds 20 meters, the loss of data packets leads to significant instruction delay (Figure 44).

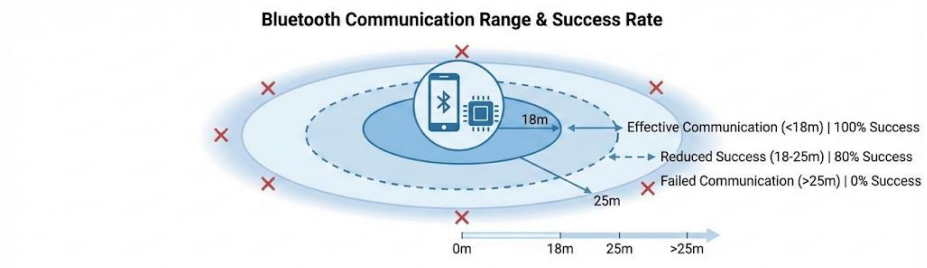


Figure 44 Schematic diagram of the hierarchical range of Bluetooth communication

To verify the obstacle avoidance function under dynamic conditions, a strict high-speed emergency braking test was conducted. The main engineering challenge is to overcome the physical inertia of the chassis. Early tests indicated that a simple power cut-off based on the variable of distance failed at high speeds, resulting in collisions due to sliding distance. To solve this problem, we implemented a speed-adaptive braking algorithm, which can monitor the speed of the car and respond to breaking in a timely manner. The experimental results confirm that whether cruising or sprinting, the robot can continuously decelerate until it comes to a complete stop, with a safety margin of 5 to 10 cm from the obstacle. This proves that the closed-loop control logic successfully compensates for mechanical delay and inertia, achieving a 100% collision avoidance rate.

Integrated Field Testing

The final stage involves conducting a comprehensive on-site test for 10 consecutive minutes in a place artificially set up to simulate a complex environment. The vehicle successfully transitioned between various terrain surfaces without stalling and could cross small obstacles. After the crash test, the vehicle body showed no cracks and the components were not loose, which proved that the anti-collision system was effective. Thermal monitoring shows that the motor driver is not overheated, and the enhanced chassis maintains structural integrity under vibration, confirming that the design of the prototype demonstration is ready.

8. Lessons Learned

Initially, we adopted a two-layer open framework architecture. This is because an open design allows for more convenient changes to the circuit. However, in the simulated "rough terrain" test, we observed that the open design lacked sufficient torsional stiffness. The vibration of the tire wheel on the uneven surface causes the fasteners to gradually loosen, and the jumpers will also become loose. In the final iteration, we shifted to a semi-closed and partitioned design. We used L-shaped aluminum brackets to connect acrylic sheets, creating a rigid box structure. This design retains convenience while significantly enhancing structural integrity and preventing component displacement during patrol maneuvers.

Our first PCB batch was rendered unusable because we overlooked the exact pin pitch and package dimensions of our existing motor driver components—this mismatch meant the drivers couldn't be correctly soldered to the board. We addressed this by re-verifying the physical specs of all in-stock parts, adjusting the PCB's component footprint layouts accordingly, and having a new PCB set fabricated. Going forward, we now confirm in-stock components' physical compatibility with preliminary PCB designs *before* initial fabrication, to avoid rework costs and delays.

In early testing, we frequently performed hot swapping, which led to damaged I/O ports, failed program burns, and even the burnout of one microcontroller core chip. These issues not only increased component replacement costs but also disrupted the testing schedule. To resolve this, we established a strict operational protocol: always power off the system first, confirm no residual voltage on the power rail, then perform jumper or programming cable adjustments, and only reapply power after the connections are secured. Moving forward, this mandatory power-off operation has been documented in our team's operation manual and shared with all members to prevent recurrent failures.

9. Future Work

Based on the testing results and the limitations identified in the current prototype, future development will focus on three key areas: mechanical reliability, communication range extension, and autonomous intelligence.

Mechanical Transmission and Terrain Adaptability. The priority is to improve the rigidity of the drive system. Current testing revealed that the fixation between the motor output shafts and the Mecanum wheels is prone to loosening under high torque. Future iterations will replace the existing couplers with high-precision clamping hubs or keyed shafts to ensure stable power transmission and prevent slippage. Additionally, to enhance the vehicle's survival capability in complex terrain, a self-extrication mechanism will be designed. This structure, potentially driven by linear actuators, will allow the chassis to lift itself or alter its ground clearance, enabling the vehicle to escape independently if it becomes trapped in mud or uneven ground.

Long-Range Control via PC Relay Architecture. To meet the requirement for border patrolling over extended distances, approximately 1 km, the communication system will be upgraded from direct Bluetooth connection to a relay-based architecture. A PC-based Ground Control Station will serve as the central relay node. In this proposed setup, the operator uses a mobile device to send commands to the PC via a local Wi-Fi network. The PC then forwards these control signals to the vehicle using high-power long-range transmission modules and receives real-time video feedback. This architecture combines the user-friendly interface of mobile control with the high power and processing capabilities of a PC, ensuring stable video streaming and low-latency control over long distances.

Intelligent Surveillance and Active Defense. The final area of improvement involves upgrading the system from a passive remote-controlled vehicle to an autonomous intelligent sentry. By leveraging the computing power of the Raspberry Pi, Edge AI algorithms, such as YOLO will be deployed to detect and classify intruders

automatically. Upon detecting a threat, the system will autonomously trigger an alarm and transmit the alert data back to the control center. Furthermore, active defense modules will be integrated. Once an intrusion is confirmed, the vehicle will be able to deploy non-lethal deterrents, such as high-intensity strobe lights or audio sirens, to warn off intruders while awaiting human intervention.

10. Conclusion

This project successfully designed, manufactured, and tested a multifunctional smart border patrol vehicle, integrating mechanical engineering, embedded control, and computer vision. The system met the primary design objectives of omnidirectional mobility, real-time surveillance, and robust remote control.

From a mechanical perspective, the chassis evolved from a simple flat-plate design to a reinforced compartmentalized structure. Finite element analysis validated this design; the static stress analysis confirmed a safety factor of 1.69 under rated loads, while explicit dynamic simulations proved the effectiveness of the spring-loaded bumper in absorbing impact energy. The application of Mecanum wheels provided the vehicle with agile omnidirectional movement capabilities essential for complex terrain navigation.

In terms of electronics and control, a dual-core distributed architecture was implemented. The LPC1768 successfully managed the underlying motion kinematics and sensor data, while the Raspberry Pi Zero 2 W handled high-bandwidth image processing. The transition from breadboard prototypes to custom-designed PCBs significantly enhanced the system's electrical stability and physical durability.

The software implementation featured a hybrid concurrency model combining multi-threading and asynchronous I/O, ensuring low-latency video streaming (approximately 100ms) via WebSockets. The development of a WeChat Mini Program provided a user-friendly interface that bridged complex hardware logic with intuitive control, supporting both discrete button inputs and analog joystick operations.

Experimental results demonstrated that the vehicle performs reliably in simulated environments. The ultrasonic obstacle avoidance system achieved a 100% success rate in emergency braking tests, and the face recognition module maintained high accuracy in varied lighting conditions. Although the autonomous PID tracking function was removed due to stability challenges, the current prototype serves as a solid foundation. The project demonstrates the feasibility of a low-cost, integrated unmanned ground vehicle for border monitoring applications.

References

[1] S. Kumar et al., “The Rise of UAV-Based Smart Surveillance: A Systematic Review of Trends and Technologies,” IEEE Access, vol. 13, pp. 181553-181575, 2025. doi:10.1109/ACCESS.2025.3621736

[2] AA Laghari, “Unmanned aerial vehicles advances in object detection and communication security review,” Computers & Graphics, vol. …, 2024.

[3] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, *Introduction to Autonomous Mobile Robots*, 2nd ed. Cambridge, MA, USA: MIT Press, 2011, pp. 13–25.

[4] W. D. Callister and D. G. Rethwisch, *Materials Science and Engineering: An Introduction*, 9th ed. New York, NY, USA: Wiley, 2013, ch. 15.

[5] R. G. Budynas and J. K. Nisbett, "Bolted and Riveted Joints," in *Shigley's Mechanical Engineering Design*, 10th ed. New York, NY, USA: McGraw-Hill Education, 2014, pp. 433–440.

[6] Z. Chen and H. Zhang, "Design of embedded mobile robot control system based on ARM and DSP," in 2011 International Conference on Computer Science and Service System (CSSS), Nanjing, China, 2011, pp. 2728-2731.

[7] NXP Semiconductors, "LPC1769/68/67/66/65/64/63: 32-bit ARM Cortex-M3 microcontroller Product Data Sheet," Rev. 9.3, Jan. 2014. [Online]. Available: https://www.nxp.com.

[8] E. Upton and G. Halfacree, Raspberry Pi User Guide, 4th ed. Chichester, U.K.: Wiley, 2016.

[9] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, Introduction to Autonomous Mobile Robots, 2nd ed. Cambridge, MA, USA: MIT Press, 2011.

[10] STMicroelectronics, "L298 - Dual Full-Bridge Driver Datasheet," 2000. [Online]. Available: https://www.st.com.

[11] H. M. D. Ali, "Performance evaluation of ultrasonic sensor HC-SR04 for distance measurement," Int. J. Eng. Res. Technol., vol. 9, no. 12, pp. 110-114, 2020.

[12] J. Borenstein and Y. Koren, "Real-time obstacle avoidance for fast mobile robots," IEEE Trans. Syst., Man, Cybern., vol. 19, no. 5, pp. 1179-1187, Sep./Oct. 1989.

[13] HC Tech, "HC-08 Bluetooth UART Communication Module User Manual," V2.0, 2016.

[14] Texas Instruments, "LM2596 SIMPLE SWITCHER® Power Converter 150-kHz 3-A Step-Down Voltage Regulator," Datasheet SNVS124D, May 2016.

[15] R. E. Bryant and D. R. O'Hallaron, *Computer systems: a programmer's perspective*, Third edition. Boston: Pearson, 2016.

[16] P. Viola and M. J. Jones, 'Robust Real-Time Face Detection', *International Journal of Computer Vision*, vol. 57, no. 2, pp. 137–154, May 2004, doi: 10.1023/B:VISI.0000013087.49260.fb.

[17] L. Ramalho, *Fluent Python: clear, concise, and effective programming*, 2nd edition. Sebastopol, California: O'Reilly Media, Inc., 2022. Accessed: Jan. 04, 2026. [Online]. Available: https://www.oreilly.com/library/view/-/9781492056348/

[18] A. Melnikov and I. Fette, 'The WebSocket Protocol', Internet Engineering Task Force, Request for Comments RFC 6455, Dec. 2011. doi: 10.17487/RFC6455.

[19] S. Josefsson, 'The Base16, Base32, and Base64 Data Encodings', Internet Engineering Task Force, Request for Comments RFC 4648, Oct. 2006. doi: 10.17487/RFC4648.

[20] 'Huabu / canvas'. Accessed: Jan. 04, 2026. [Online]. Available:

https://developers.weixin.qq.com/miniprogram/dev/component/canvas.html

Appendix

A. Command Manual

Single Commands

- These commands start with `#`, in the format of `# A`.
- `U/u`: Start/Stop moving forward
- `D/d`: Start/Stop moving backward
- `L/l`: Start/Stop moving left
- `R/r`: Start/Stop moving right
- `B/b`: Start/Stop rotating counter-clockwise
- `E/e`: Start/Stop rotating clockwise

Multi-Parameter Commands

These commands start with `:`, in the format of `: V 0.71 0.71`.

- `V v_x v_y`: Send velocity vector $\vec{v} = (v_x, v_y)$; parameters are floating-point numbers.
- `SM JK/BT`: Switch to Joystick (`JK`) / Button (`BT`) mode.
- `SPD speed`: Set the base speed to `speed`; parameter is an integer.
- `SVO UD/LR deg`: Adjust the Up-Down (UD) or Left-Right (LR) servo to the specified angle. `deg` is the servo angle ranging from $-90°$ to $90°$.
- `CAM SHOT/REC/END`: Raspberry Pi Camera: Take photo / Start recording / End recording.
- `AI 1/0`: Enable/Disable portrait (object) recognition.
- `TRK 1/0`: Start/Stop object tracking.
- `OBJ x_c y_c a b`: Describe the position and the dimension of the object, where (x_c, y_c) indicates the coordinates of the centroid of the rectangle and a, b are the lengths in the x and y direction respectively.

B. Crucial Code

Due to length limitations, the following only demonstrates some of the important code. All the code of this program are uploaded to *GitHub.* You can see the details of the program to view the complete project and perform local tests.

- Motion control of the car (LPC1768): https://github.com/MEshooter/mbed_mecanum_car
- Vision Module (Raspberry Pi): https://github.com/MEshooter/berryeye
- Controller on the mobile (WeChat Mini Program): https://github.com/MEshooter/bluetooth_controller

B.1. Motion Control (LPC1768)

This part only displays the code of `main.cpp` and `MotionControl` class. The full project can be visited via: https://github.com/MEshooter/mbed_mecanum_car

Main Program

```cpp
// main.cpp
#ifdef __ARM_FP
#undef  __ARM_FP
#endif

#include "mbed.h"
#include "MotionControl.h"
#include "Vector.hpp"
#include "StdMotor.h"
#include "Ultrasonic.hpp"

#include <cstring>
#include <cstdio>
#include <cstdint>
#include <deque>

const int WIDTH = 320, HEIGHT = 240, A = 320 * 240;

const int period = 50; // unit: us
StdMotor BR(p21, p19, p18, period);
StdMotor BL(p22, p17, p16, period);
StdMotor FR(p23, p15, p14, period);
StdMotor FL(p24, p13, p12, period);
MotionControl carMotion(FL, FR, BL, BR);

// Commands read from the bluetooth UART
struct CmdInfo{
    char text[32];
    int len;
    CmdInfo() {memset(text, 0, sizeof(text)); len = 0;}
};

// Activated status for variable flag
const uint32_t BT_READABLE = (1 << 0);
const uint32_t RPI_READABLE = (1 << 1);
const int speedGrade = 7;

bool sonicSwitch = 0;

BufferedSerial pc(USBTX, USBRX);
BufferedSerial bt(p9, p10);
BufferedSerial rpi(p28, p27);
Ultrasonic sonar(p29, p30);
```

```
EventFlags flag;
// The thread used to read the command from the bluetooth UART
Thread inputThread;
// Mailbox is used to transfer messages between threads,
// ensuring the safety of threads.
Mail<CmdInfo, 16> cmdMail;

void btInterrupt();
void rpiInterrupt();
void inputProcess();
void cmdProcess(const char*, int len);

int main() {
    // hb_timer.start();
    // Register an interrupt without blocking for the bluetooth UART
    bt.sigio(callback(btInterrupt));
    bt.set_blocking(0);
    bt.set_baud(9600);

    rpi.sigio(callback(rpiInterrupt));
    rpi.set_blocking(1);
    rpi.set_baud(115200);

    inputThread.start(callback(inputProcess));

    printf("Successfully started.\n");
    while (true) {
        // send_heartbeat();
        float dist = sonar.getDistance();
        float distLimit = 15 * speedGrade - 70;
        bool isLocked = (sonicSwitch && dist < distLimit && dist > 2.0);

        FL.setForwardLock(isLocked);
        FR.setForwardLock(isLocked);
        BL.setForwardLock(isLocked);
        BR.setForwardLock(isLocked);

        osEvent evt = cmdMail.get(0);
        // If there is messages in the mailbox, take out it and parse the texts.
        if(evt.status == osEventMail){
            CmdInfo* msg = (CmdInfo*)evt.value.p;
            cmdProcess(msg->text, msg->len);
            cmdMail.free(msg);
            // When the processing is finished, release the memory of the mail
```

```
        }
    }
}

// The interrupt will be triggered,
// when there is a message on the bluetooth UART.
void btInterrupt() {
    // Set the flag to be activated,
    // which will let inputProcess to read the message.
    flag.set(BT_READABLE);
}

void rpiInterrupt() {
    flag.set(RPI_READABLE);
}

void inputProcess() {
    const int BUF_SIZE = 32;

    char btBuf[BUF_SIZE], rpiBuf[BUF_SIZE], ch;
    int btPtr = 0, rpiPtr = 0;

    auto charProcess = [&] (char buffer[], int &ptr) {
        // Input processing ends with the string stored in the mailbox
        // when reaching the end of the string
        // or the buffer overflows.
        // Otherwise, just put the character into the buffer.
        if(ch == '\n' || ch == '\r' || ch == '\0' || ptr == 31) {
            buffer[ptr] = '\0';
            CmdInfo *msg = cmdMail.alloc();
            if(msg){
                std::memcpy(msg->text, buffer, ptr + 1);
                msg->len = ptr;
                cmdMail.put(msg);
            }
            memset(buffer, 0, BUF_SIZE);
            ptr = 0;

        }
        else buffer[ptr++] = ch;
    };

    while(true) {
        flag.wait_any(BT_READABLE | RPI_READABLE);
```

```
        while(bt.readable()) {
            bt.read(&ch, 1);
            charProcess(btBuf, btPtr);
        }

        while(rpi.readable()) {
            rpi.read(&ch, 1);
            charProcess(rpiBuf, rpiPtr);
        }
    }
}

void cmdProcess(const char* text, int len){
    printf("Len: %d, %s\r\n", len, text);
    bt.write(text, strlen(text));
    fflush(stdout);
    // The detailed instructions on the commands can be referred in the mannual.
    // '#': Commands with s single alphabet
    if(text[0] == '#'){
        char command = text[2];
        if(isupper(command)){
            Direction tmp(0, 0, 0, 0);
            auto update = [&]{
                carMotion.addDir(tmp);
                carMotion.updateMotion();
            };
            switch(command){
                case 'U': tmp.f = 1; update(); break;
                case 'D': tmp.b = 1; update(); break;
                case 'L': tmp.l = 1; update(); break;
                case 'R': tmp.r = 1; update(); break;
                case 'B': carMotion.setRotation(-1); break;
                case 'E': carMotion.setRotation(1); break;
                case 'S':
                    sonicSwitch=!sonicSwitch;
                    char str[32];
                    sprintf(str, "Ultrasonic Mode: %d\n", sonicSwitch);
                    bt.write(str, strlen(str));
                    break;
            }
        }
        if(islower(command)){
            Direction tmp(1, 1, 1, 1);
```

```
        auto update = [&]{
            carMotion.delDir(tmp);
            carMotion.updateMotion();
        };
        switch(command){
            case 'u': tmp.f = 0; update(); break;
            case 'd': tmp.b = 0; update(); break;
            case 'l': tmp.l = 0; update(); break;
            case 'r': tmp.r = 0; update(); break;
            case 'b':
            case 'e': carMotion.setRotation(0); break;
            case 's': break;
        }
    }
}
// ':': Commands with a group of phrases
if(text[0] == ':') {
    char command[16];
    sscanf(text + 2, "%15s", command);
    if(strcmp(command, "SPD") == 0) {
        int speed;
        sscanf(text + 6, "%d", &speed);
        carMotion.setBaseSpeed(5 + speed * 0.5);
    }
    if(strcmp(command, "V") == 0) {
        double vx, vy;
        sscanf(text + 4, "%lf%lf", &vx, &vy);
        carMotion.updateMotion(Vector<double>(vx, vy));
    }
    if(strcmp(command, "SM") == 0) {
        char mode[8];
        sscanf(text + 5, "%7s", mode);
        if(strcmp(mode, "JS") == 0) carMotion.changeMode(JOYSTICK);
        if(strcmp(mode, "BT") == 0) carMotion.changeMode(BUTTON);
    }
    if(strcmp(command, "OBJ") == 0) {
        // double x, y, a, b;
        // sscanf(text + 6, "%lf%lf%lf%lf", &x, &y, &a, &b);
        // double x_c = WIDTH / 2.0, y_c = HEIGHT / 2.0, area =  4 * a * b;
        // lastState.emplace_back(x, y, area);
        // while(lastState.size() > 5) lastState.pop_front();
    }

    bool isRpi = ((strcmp(command, "SVO") == 0) ||
```

```
            (strcmp(command, "CAM") == 0) ||
            (strcmp(command, "TRK") == 0) ||
            (strcmp(command, "AI") == 0));

        if(isRpi) {
            char msg[32];
            sprintf(msg, "Sent \"%s\"\n", text);
            bt.write(msg, strlen(msg));
            memset(msg, 0, sizeof(msg));
            sprintf(msg, "%s\n", text);
            rpi.write(msg, strlen(msg));
        }
    }
}
```

Motion Control

```
/**
 * The library is designed for motion controlling of the vehicle.
 * BUTTON mode allows the car to move in four directions.
 * JOYSTICK mode allows the car to move in all directions.
 * The moving direction of the car is based on a velocity vector,
 * and the magnitude of the velocity is based on the variable baseSpeed.
 */
// MotionControl.h
#ifndef MOTIONCONTROL_H
#define MOTIONCONTROL_H

#ifdef __ARM_FP
#undef  __ARM_FP
#endif

#include "mbed.h"
#include "StdMotor.h"
#include "Vector.hpp"

enum ControlMode{
    BUTTON, JOYSTICK
};

// Struct Direction is for the direction control in BUTTON mode
struct Direction{
    bool f, b, l, r;
    Direction(bool _f = 0, bool _b = 0, bool _l = 0, bool _r = 0);
    Direction& operator |= (const Direction &t){
        f |= t.f; b |= t.b; l |= t.l; r |= t.r;
```

```
        return *this;
    }
    Direction& operator &= (const Direction &t){
        f &= t.f; b &= t.b; l &= t.l; r &= t.r;
        return *this;
    }
    friend Direction operator | (const Direction&, const Direction&);
    friend Direction operator & (const Direction&, const Direction&);
};

Vector<double> dir2Vector(const Direction&);

class MotionControl{
private:
    StdMotor &FL, &FR, &BL, &BR;
    double baseSpeed;
    int w; // +1: cw -1: ccw
    ControlMode mode;
    Direction dir;
    Vector<double> v;
public:
    MotionControl(StdMotor&, StdMotor&, StdMotor&, StdMotor&);
    ControlMode getMode();
    void changeMode(ControlMode);
    void addDir(const Direction&);
    void delDir(const Direction&);
    void setRotation(const int&);
    void setBaseSpeed(const double&);
    void updateMotion(const Vector<double>& _v = Vector<double>());
    void stop();
};

#endif

// MotionControl.cpp
#include "MotionControl.h"

Direction::Direction(bool _f, bool _b, bool _l, bool _r):
    f(_f), b(_b), l(_l), r(_r) {}

Direction operator | (const Direction& a, const Direction& b) {
    Direction res = a;
    res |= b;
    return res;
```

```
}

Direction operator & (const Direction& a, const Direction& b) {
    Direction res = a;
    res &= b;
    return res;
}

Vector<double> dir2Vector(const Direction &dir) {
    double v_x = 0, v_y = 0;
    if(dir.f) v_y += 1;
    if(dir.b) v_y -= 1;
    if(dir.l) v_x -= 1;
    if(dir.r) v_x += 1;
    Vector<double> res(v_x, v_y);
    return res.unit();
}

MotionControl::MotionControl(StdMotor& _FL, StdMotor& _FR, StdMotor& _BL, StdMotor& _BR):
    FL(_FL), FR(_FR), BL(_BL), BR(_BR) {
    baseSpeed = 50;
    dir = Direction();
    mode = BUTTON;
    v = {0, 0};
    w = 0;
}

ControlMode MotionControl::getMode() {
    return mode;
}

void MotionControl::changeMode(ControlMode _mode) {
    mode = _mode;
}

void MotionControl::addDir(const Direction &_d) {
    dir |= _d;
}

void MotionControl::delDir(const Direction &_d) {
    dir &= _d;
}

void MotionControl::setRotation(const int& _w) {
```

```
    if(w != 0 && _w != 0) w = 0;
    else w = _w;
    updateMotion(v);
}

void MotionControl::setBaseSpeed(const double& speed) {
    baseSpeed = std::min(std::max(5.0, speed), 10.0) * 10;
}

void MotionControl::updateMotion(const Vector<double>& _v) {
    if(mode == BUTTON) v = dir2Vector(dir);
    else v = _v.unit();

    Vector<double> V = v * (double)baseSpeed;
    double W = baseSpeed * w;

    char prt[64];
    sprintf(prt, "VELOCITY: %.2f %.2f %.2f\n", V.x, V.y, W);
    printf("%s", prt);
    fflush(stdout);

    double speedFL = V.y + V.x + W;
    double speedFR = V.y - V.x - W;
    double speedBL = V.y - V.x + W;
    double speedBR = V.y + V.x - W;

    sprintf(prt, "SPEED: %.2f %.2f %.2f %.2f\n", speedFL, speedFR, speedBL, speedBR);
    printf("%s", prt);
    fflush(stdout);

    FL.setSpeed(speedFL);
    FR.setSpeed(speedFR);
    BL.setSpeed(speedBL);
    BR.setSpeed(speedBR);
}

void MotionControl::stop() {
    w = 0;
    dir = {0, 0, 0, 0};
    updateMotion(Vector<double>());
}
```

B.2. Vision Module (Raspberry Pi)

This part only lists the Python programs, which does not include supported files (such as materials used by Haar Cascade Classifier). Visit the link to view the whole project: https://github.com/MEshooter/berryeye

Main Program

```python
# main.py
import asyncio, websockets, socket, json
from gpiozero import AngularServo
from gpiozero.pins.pigpio import PiGPIOFactory
from time import sleep
from camera_module import CameraStreamer
from serial_module import SerialManager

# --- Configuration ---
WS_PORT = 8765
SERIAL_PORT = '/dev/serial0'
BAUDRATE = 115200

# --- Hardware Initialization ---
# Initialize GPIO Factory
factory = PiGPIOFactory()

# Common servo configuration to avoid repetition
servo_cfg = {
    "pin_factory": factory, "initial_angle": 0,
    "min_angle": -90, "max_angle": 90,
    "min_pulse_width": 0.0005, "max_pulse_width": 0.0025
}

# Initialize Servos (Left-Right on GPIO 12, Up-Down on GPIO 13)
servo_lr = AngularServo(12, **servo_cfg)
servo_ud = AngularServo(13, **servo_cfg)

# Initialize Camera
camera = CameraStreamer()

# --- Serial Data Handler ---
def on_serial_received(data):
    """
    Callback triggered when serial data is received from MCU.
    Protocol format example: ': SVO LR 90' or ': AI 1'
    """
    print(f"Received: {data.strip()}")
    uart.send(f"RPi Got: {data.strip()}") # Acknowledge
```

```python
parts = data.split()
if not parts: return

cmd_id = parts[0]

# Handle 'Heartbeat' or Sleep command
if cmd_id == "#":
    sleep(1e-6)

# Handle Action Commands
elif cmd_id == ":":
    if len(parts) < 2: return
    cmd = parts[1]

    try:
        # Servo Control: : SVO [DIR] [ANGLE]
        if cmd == "SVO" and len(parts) >= 4:
            direction, angle = parts[2], int(parts[3])
            if direction == "LR": servo_lr.angle = angle
            elif direction == "UD": servo_ud.angle = angle

        # AI Toggle: : AI [1/0]
        elif cmd == "AI" and len(parts) >= 3:
            camera.set_ai_status(int(parts[2]) == 1)

        # Tracking Toggle: : TRK [1/0]
        elif cmd == "TRK" and len(parts) >= 3:
            camera.set_trk_status(int(parts[2]) == 1)

        # Camera Operations: : CAM [OP]
        elif cmd == "CAM" and len(parts) >= 3:
            op = parts[2]
            if op == "SHOT": camera.take_photo()
            elif op == "REC": camera.start_recording()
            elif op == "END": camera.stop_recording()

        else:
            print("Invalid Command Format")
    except ValueError:
        print("Data Type Error")
else:
    print(f"[*] Unknown Data: {data}")
```

```python
# Initialize Serial Manager
uart = SerialManager(port=SERIAL_PORT, baudrate=BAUDRATE, data_callback=on_serial_received)

# --- Utilities ---
def get_host_ip():
    """Get the local IP address of the Raspberry Pi."""
    try:
        with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
            s.connect(('8.8.8.8', 80))
            return s.getsockname()[0]
    except:
        return "127.0.0.1"

# --- WebSocket Handler ---
async def ws_handler(ws):
    """Handles video streaming and AI coordinate feedback."""
    client_ip = ws.remote_address[0]
    print(f"[+] Client Connected: {client_ip}")

    # Notify camera module to wake up
    camera.client_connected()

    last_sent_id = -1

    try:
        while True:
            # 1. Video Streaming
            # Only send if frame data exists and ID has incremented
            current_id, frame_data = camera.get_frame()
            if frame_data and current_id > last_sent_id:
                await ws.send(frame_data)
                last_sent_id = current_id

            # 2. AI Tracking Feedback
            # If AI is active, calculate target center and send to Serial
            if camera.ai_active:
                targets = []
                # Use lock to safely access shared detection data
                with camera.lock_ai:
                    targets = list(camera.detected_faces)

                if targets:
                    x, y, w, h = targets[0]
                    cx, cy = int(x + w / 2), int(y + h / 2)
```

```
                    # Send coordinates: OBJ:CX,CY,HW,HH
                    uart.send(f"OBJ:{cx},{cy},{w/2},{h/2}")

            # Control loop rate (~25 FPS)
            await asyncio.sleep(0.04)

    except websockets.ConnectionClosed:
        print("[-] Client Disconnected")
    except Exception as e:
        print(f"[!] WS Error: {e}")
    finally:
        # Notify camera module to sleep/cleanup
        camera.client_disconnected()

# --- Main Entry Point ---
async def main():
    # Start background threads
    camera.start()
    uart.start()
    uart.send("RPi System Started.\n")

    ip = get_host_ip()
    print("=" * 40)
    print(f"[*] WebSocket Server: ws://{ip}:{WS_PORT}")
    print(f"[*] Serial Port: {SERIAL_PORT} @ {BAUDRATE}")
    print("=" * 40)

    # Start WebSocket Server
    async with websockets.serve(ws_handler, "0.0.0.0", WS_PORT):
        await asyncio.Future() # Run forever

if __name__ == "__main__":
    try:
        asyncio.run(main())
    except KeyboardInterrupt:
        print("\nStopping System...")
        camera.stop()
        uart.stop()
```

Camera Module

```
# camera_module.py
```

```python
import time, threading, base64, cv2, os, asyncio, numpy as np
from picamera2 import Picamera2
from datetime import datetime

class CameraStreamer:
    def __init__(self, width=480, height=320, target_fps=15, quality=70, ws_clients_ref=None):
        # Stream Config
        self.width, self.height, self.target_fps, self.quality = width, height, target_fps, quality
        self.ws_clients_ref = ws_clients_ref
        self.active_clients = 0

        # Lock Config
        self.lock_clients, self.lock_frame, self.lock_ai, self.lock_ai_img, self.cam_lock,
self.recording_lock = [threading.Lock() for _ in range(6)]

        # AI & State Config
        self.ai_size = (320, 240)
        self.ai_active = self.trk_active = self.running = self.is_recording = False
        self.paths = {"face": "haarcascade_frontalface_default.xml", "body": "haarcascade_fullbody.xml"}

        # Data Containers
        self.frame_id, self.curr_frame_b64, self.video_filename = 0, None, ""
        self.detected_faces, self.detected_bodies, self.primary_targets = [], [], []
        self.face_cascade = self.body_cascade = self.video_writer = self.picam2 = self.ai_img_data = None

        # Setup
        self._load_models()
        for d in ["photos", "videos"]: os.makedirs(d, exist_ok=True)

    def _load_models(self):
        """Load OpenCV Haar cascades."""
        load = lambda p: cv2.CascadeClassifier(p) if os.path.exists(p) else None
        self.face_cascade, self.body_cascade = load(self.paths["face"]), load(self.paths["body"])
        self.ai_loaded = bool(self.face_cascade or self.body_cascade)
        print(f"[*] Models Loaded: Face={bool(self.face_cascade)}, Body={bool(self.body_cascade)}")

    def client_connected(self):
        with self.lock_clients: self.active_clients += 1
        print(f"[*] Client connected. Total: {self.active_clients}")

    def client_disconnected(self):
        with self.lock_clients: self.active_clients = max(0, self.active_clients - 1)
        print(f"[*] Client disconnected. Total: {self.active_clients}")
```

```
    def set_ai_status(self, status: bool):
        self.ai_active = status
        if not status:
            with self.lock_ai: self.detected_faces = self.detected_bodies = self.primary_targets = []
        print(f"[*] AI Status: {status}")

    def set_trk_status(self, status: bool):
        self.trk_active = status
        if status: self.set_ai_status(True)
        print(f"[*] Tracking Status: {status}")

    def start(self):
        if self.running: return
        self.running = True
        self._init_hw()

        self.thread_cam = threading.Thread(target=self._cam_loop, daemon=True)
        self.thread_cam.start()

        if self.ai_loaded:
            self.thread_ai = threading.Thread(target=self._ai_loop, daemon=True)
            self.thread_ai.start()
        print("[*] Streamer started.")

    def stop(self):
        self.running = False
        for t in [self.thread_cam, self.thread_ai]:
            if t: t.join(timeout=1)
        self._release_hw()
        self.stop_recording()
        print("[*] Streamer stopped.")

    def get_frame(self):
        with self.lock_frame: return self.frame_id, self.curr_frame_b64

    @property
    def detected_faces_shim(self):
        with self.lock_ai: return list(self.primary_targets)

    def _init_hw(self):
        with self.cam_lock:
            if self.picam2: return
            try:
                self.picam2 = Picamera2()
```

```
            # Config: keeping RGB888 as per your "other functions normal" report
            cfg = self.picam2.create_video_configuration(
                main={"size": (self.width, self.height), "format": "RGB888"},
                lores={"size": self.ai_size, "format": "YUV420"})
            self.picam2.configure(cfg)
            try: self.picam2.set_controls({"AeFlickerMode": 1})
            except: pass
            self.picam2.start()
            print("[*] HW Initialized.")
        except Exception as e: print(f"[!] Init Error: {e}"); self.picam2 = None

    def _release_hw(self):
        with self.cam_lock:
            if self.picam2:
                try: self.picam2.stop(); self.picam2.close()
                except: pass
                self.picam2 = None
                print("[*] HW Released.")

    # --------------------------
    # Thread 1: Capture & Display
    # --------------------------
    def _cam_loop(self):
        sx, sy = self.width / self.ai_size[0], self.height / self.ai_size[1]

        while self.running:
            with self.lock_clients: active = self.active_clients > 0 or self.is_recording
            if not active:
                if self.picam2: self._release_hw()
                time.sleep(0.1); continue

            if not self.picam2: self._init_hw()
            if not self.picam2: time.sleep(1); continue

            try:
                t0 = time.time()
                img = self.picam2.capture_array("main")
                if img is None: continue

                if self.ai_loaded and self.ai_active:
                    lores = self.picam2.capture_array("lores")
                    gray = cv2.cvtColor(lores, cv2.COLOR_YUV2GRAY_I420)
                    with self.lock_ai_img: self.ai_img_data = gray
```

```
                with self.lock_ai:
                    # Draw boxes (Colors: Red=0,0,255 / Green=0,255,0)
                    # Note: On RGB data, (0,0,255) draws Blue.
                    # If you want real Red on RGB, use (255,0,0).
                    # But keeping logic consistent with your request.
                    draw_list = [(self.detected_faces, (0,0,255), ""), (self.detected_bodies, (0,255,0),
"Body")]
                    for targets, color, lbl in draw_list:
                        for (x, y, w, h) in targets:
                            mx, my, mw, mh = int(x*sx), int(y*sy), int(w*sx), int(h*sy)
                            cv2.rectangle(img, (mx, my), (mx+mw, my+mh), color, 2)
                            if lbl: cv2.putText(img, lbl, (mx, my-5), cv2.FONT_HERSHEY_SIMPLEX, 0.5, color,
1)

                # Recording Logic - FIX: Removed cvtColor RGB2BGR
                with self.recording_lock:
                    if self.is_recording and self.video_writer:
                        # Directly write 'img' to match stream colors
                        self.video_writer.write(img)

                # Streaming Logic - Encodes 'img' directly
                ok, buf = cv2.imencode('.jpg', img, [cv2.IMWRITE_JPEG_QUALITY, self.quality])
                if ok:
                    with self.lock_frame:
                        self.curr_frame_b64 = base64.b64encode(buf).decode('utf-8')
                        self.frame_id += 1

                wait = (1.0 / self.target_fps) - (time.time() - t0)
                if wait > 0: time.sleep(wait)
            except Exception as e:
                print(f"[!] Stream Error: {e}"); self._release_hw(); time.sleep(2)

    # -------------------------
    # Thread 2: AI Processing
    # -------------------------
    def _ai_loop(self):
        print("[*] AI thread started.")
        while self.running:
            try:
                with self.lock_ai_img: img_in = self.ai_img_data
                if img_in is not None:
                    faces, bodies, primary = [], [], []

                    if self.trk_active:
```

```
                        if self.body_cascade:
                            res = self.body_cascade.detectMultiScale(img_in, 1.1, 3, minSize=(60, 120))
                            bodies = list(res) if not isinstance(res, tuple) else []
                        primary = bodies
                    else:
                        if self.face_cascade:
                            res = self.face_cascade.detectMultiScale(img_in, 1.1, 4, minSize=(20, 20))
                            faces = list(res) if not isinstance(res, tuple) else []
                        primary = []

                    with self.lock_ai:
                        self.detected_faces, self.detected_bodies, self.primary_targets = faces, bodies,
primary
                time.sleep(0.05)
            except Exception as e: print(f"[!] AI Error: {e}"); time.sleep(1)

    def take_photo(self):
        if not self.picam2: return print("[!] Photo failed: Cam not ready.")
        fname = os.path.join("photos", f"IMG_{datetime.now().strftime('%Y%m%d_%H%M%S')}.jpg")

        with self.cam_lock:
            try:
                img = self.picam2.capture_array("main")
                if self.ai_active:
                    with self.lock_ai: targets = list(self.detected_faces)
                    sx, sy = self.width / self.ai_size[0], self.height / self.ai_size[1]
                    for (x, y, w, h) in targets:
                        cv2.rectangle(img, (int(x*sx), int(y*sy)), (int((x+w)*sx), int((y+h)*sy)), (0, 0,
255), 2)

                # Photo Save - FIX: Removed cvtColor RGB2BGR
                # Saving 'img' directly as it is consistent with the stream
                cv2.imwrite(fname, img)

                # Send to Phone - Encodes 'img' directly
                ok, buf = cv2.imencode('.jpg', img)
                if ok and self.ws_clients_ref:
                    msg = "PHOTO:" + base64.b64encode(buf).decode('utf-8')
                    asyncio.run_coroutine_threadsafe(self._broadcast(msg), asyncio.get_event_loop())
                print(f"[*] Photo saved: {fname}")
            except Exception as e: print(f"[!] Photo Error: {e}")

    async def _broadcast(self, msg):
        if self.ws_clients_ref: await asyncio.gather(*[c.send(msg) for c in self.ws_clients_ref])
```

```
    def start_recording(self):
        with self.recording_lock:
            if self.is_recording or not self.picam2: return
            try:
                self.video_filename = os.path.join("videos",
f"REC_{datetime.now().strftime('%Y%m%d_%H%M%S')}.avi")
                fourcc = cv2.VideoWriter_fourcc(*'XVID')
                self.video_writer = cv2.VideoWriter(self.video_filename, fourcc, self.target_fps,
(self.width, self.height))
                self.is_recording = self.video_writer.isOpened()
                print(f"[*] Recording: {self.video_filename}")
            except Exception as e: print(f"[!] Record Error: {e}"); self.is_recording = False

    def stop_recording(self):
        with self.recording_lock:
            if self.video_writer: self.video_writer.release()
            self.video_writer, self.is_recording = None, False
            print("[*] Recording stopped.")
```

Serial Module

```
# serial_module.py

import serial, threading, time

class SerialManager:
    def __init__(self, port="/dev/ttyAMA0", baudrate=115200, data_callback=None):
        self.port, self.baud, self.callback = port, baudrate, data_callback
        self.ser, self.running, self.thread = None, False, None

    def start(self):
        """Initialize serial port and start the background reading thread."""
        try:
            # timeout=1.0 allows the thread to wake up periodically to check self.running
            # while keeping CPU usage at 0% during the wait.
            self.ser = serial.Serial(self.port, self.baud, timeout=1.0)
            self.running = True
            self.thread = threading.Thread(target=self._read_loop, daemon=True)
            self.thread.start()
            print(f"[*] Serial started (blocking mode): {self.port}")
        except Exception as e: print(f"[!] Serial open failed: {e}")

    def stop(self):
```

```
        """Stop the loop and release the serial port."""
        self.running = False
        if self.ser and self.ser.is_open: self.ser.close()
        if self.thread: self.thread.join(timeout=1)

    def send(self, data_str):
        """Send a string to the serial port followed by a newline."""
        if self.ser and self.ser.is_open:
            try:
                self.ser.write(f"{data_str}\n".encode('utf-8'))
            except Exception as e: print(f"[!] Serial send error: {e}")

    def _read_loop(self):
        """Internal loop for blocking data retrieval."""
        while self.running:
            try:
                if self.ser.is_open:
                    # readline() blocks until a '\n' is received or timeout (1s) occurs
                    line = self.ser.readline().decode('utf-8', errors='ignore').strip()
                    if line:
                        print(f"[MCU -> RPi]: {line}")
                        if self.callback: self.callback(line)
            except (serial.SerialException, AttributeError):
                break # Port closed or disconnected
            except Exception as e:
                print(f"[!] Serial read error: {e}")
                time.sleep(1) # Delay to prevent rapid error logging
```

B.3. WeChat Mini Program

This part only includes the JavaScript file. The whole project is uploaded on:
https://github.com/MEshooter/bluetooth_controller

```
// pages/index/index.js
const locales = require('../../utils/locales.js');

Page({
  data: {
    curLang: 'zh', t: locales.TRANS['zh'],
    isConnected: false, isScanning: false, isConnecting: false, devices: [],
    connectedDeviceId: '', serviceId: '', characteristicId: '', connectedName: '',
    inputText: '', isJoystickMode: false,
    stickX: 0, stickY: 0, joystickRadius: 0, centerX: 0, centerY: 0,
    lastSendTime: 0, speed: 5, logList: [], scrollTop: 0,
    isVideoMode: false, videoLoading: false,
```

```
  camAngleLR: 0, camAngleUD: 0, isRecording: false, isAI: false, isTRK: false
},

// Instance variables
canvasNode: null, canvasCtx: null, socketTask: null, camTimer: null,

onLoad() {
  // 1. Load Language (Using new API)
  try {
    const sysLang = wx.getAppBaseInfo().language.toLowerCase();
    const lang = sysLang.includes('en') ? 'en' : 'zh';
    this.setData({ curLang: lang, t: locales.TRANS[lang] });
    wx.setNavigationBarTitle({ title: locales.TRANS[lang].title });
  } catch (e) { console.error("Lang Init Error:", e); }

  // 2. Calculate Video Window Size
  try {
    const win = wx.getWindowInfo();
    const vW = (320 / 750) * win.windowWidth, vH = (240 / 750) * win.windowWidth;
    this.setData({ videoX: (win.windowWidth - vW) / 2, videoY: (win.windowHeight - vH) / 2 - 80 });
  } catch (e) { console.error("Size Calc Error:", e); }

  // 3. Init Bluetooth
  wx.openBluetoothAdapter({ fail: (e) => console.log("BLE Init Fail:", e) });
},

toggleLang() {
  const newLang = this.data.curLang === 'zh' ? 'en' : 'zh';
  this.setData({ curLang: newLang, t: locales.TRANS[newLang] });
  wx.vibrateShort();
  wx.setNavigationBarTitle({ title: locales.TRANS[newLang].title });
},

addLog(msg, type = 'sys') {
  const list = this.data.logList;
  if (list.length > 50) list.shift();
  list.push({ time: new Date().toTimeString().split(' ')[0], type, msg });
  this.setData({ logList: list, scrollTop: list.length * 100 });
},

sendData(str) {
  this.addLog(`Tx> ${str}`, 'tx');
  if (this.data.connectedDeviceId === 'DEBUG' || !this.data.isConnected) return;
  const s = str + '\n', buf = new ArrayBuffer(s.length), v = new DataView(buf);
```

```
    for (let i = 0; i < s.length; i++) v.setUint8(i, s.charCodeAt(i));
    wx.writeBLECharacteristicValue({
      deviceId: this.data.connectedDeviceId, serviceId: this.data.serviceId,
      characteristicId: this.data.characteristicId, value: buf,
      fail: (e) => this.addLog(`Err: ${e.errMsg}`, 'sys')
    });
  },

  // --- Bluetooth Logic ---
  startScan() {
    this.setData({ isScanning: true, devices: [], logList: [] });
    wx.closeBluetoothAdapter({ complete: () => setTimeout(this.initBluetooth, 200) });
  },
  initBluetooth() {
    wx.openBluetoothAdapter({
      success: () => this.startDiscovery(),
      fail: () => { this.setData({isScanning:false}); wx.showToast({title: this.data.t.ble_off,
icon:'none'}); }
    });
  },
  startDiscovery() {
    wx.startBluetoothDevicesDiscovery({ allowDuplicatesKey: false, success: () => {
      wx.onBluetoothDeviceFound(res => {
        res.devices.forEach(d => {
          if (d.name && d.name.trim() && !this.data.devices.some(x => x.deviceId === d.deviceId)) {
            this.setData({ devices: [...this.data.devices, d] });
          }
        });
      });
    }});
  },
  connectDevice(e) {
    if (this.data.isConnecting) return;
    const { id, name } = e.currentTarget.dataset;
    this.setData({ isConnecting: true }); wx.stopBluetoothDevicesDiscovery();
    wx.showLoading({title: this.data.t.connect});

    wx.createBLEConnection({
      deviceId: id, timeout: 10000,
      success: () => { this.setData({connectedDeviceId: id, connectedName: name, isConnecting: false});
this.getServices(id); },
      fail: (e) => { wx.hideLoading(); this.setData({isConnecting:false});
this.addLog(`${this.data.t.conn_fail}: ${e.errMsg}`); }
    });
```

```
  },
  getServices(devId) {
    wx.getBLEDeviceServices({ deviceId: devId, success: (res) => {
      const s = res.services.find(i => i.uuid.toUpperCase().includes('FFE0'));
      if (s) { this.setData({serviceId: s.uuid}); this.getCharacteristics(devId, s.uuid); }
    }});
  },
  getCharacteristics(devId, svcId) {
    wx.getBLEDeviceCharacteristics({
      deviceId: devId, serviceId: svcId,
      success: (res) => {
        const c = res.characteristics.find(i => i.uuid.toUpperCase().includes('FFE1'));
        if (c) {
          this.setData({ characteristicId: c.uuid, isConnected: true });
          wx.hideLoading();
          this.addLog(this.data.t.conn_ok, 'sys');
          setTimeout(() => {
            wx.notifyBLECharacteristicValueChange({
              state: true, deviceId: devId, serviceId: svcId, characteristicId: c.uuid,
              success: () => { this.addLog(this.data.t.listen_on, 'sys'); this.startRx(); },
              fail: (e) => this.addLog(`${this.data.t.listen_fail}: ${e.errMsg}`, 'sys')
            });
          }, 500);
        }
      }
    });
  },
  startRx() {
    wx.onBLECharacteristicValueChange((res) => {
      let str = "", v = new DataView(res.value);
      for (let i = 0; i < v.byteLength; i++) str += String.fromCharCode(v.getUint8(i));
      this.addLog(`Rx< ${str.replace(/\n|\r/g, "")}`, 'rx');
    });
  },
  disconnect() {
    if (this.data.connectedDeviceId && this.data.connectedDeviceId!=='DEBUG')
      wx.closeBLEConnection({deviceId: this.data.connectedDeviceId});
    this.setData({isConnected:false, devices:[], isConnecting:false});
  },

  // --- Input & Control ---
  handleBtnPress(e) { if(this.data.isConnected) this.sendData(`#
${e.currentTarget.dataset.char.toUpperCase()}`); },
```

```
  handleBtnRelease(e) { if(this.data.isConnected) this.sendData(`#
${e.currentTarget.dataset.char.toLowerCase()}`); },
  sendInputCmd() { if(this.data.inputText) this.sendData(this.data.inputText); },
  handleSpeedChange(e) { this.setData({speed: e.detail.value}); this.sendData(`: SPD
${e.detail.value}`); },
  handleInput(e) { this.setData({ inputText: e.detail.value }); },
  enterDebugMode() {
    this.setData({ isConnected: true, connectedName: this.data.t.debug_name, connectedDeviceId: 'DEBUG',
isScanning: false });
    this.addLog(this.data.t.debug_start); wx.stopBluetoothDevicesDiscovery();
    setTimeout(() => { if(this.data.isJoystickMode) this.initJoystick(); }, 200);
  },

  // --- Joystick Logic ---
  toggleMode() {
    const js = !this.data.isJoystickMode;
    this.setData({ isJoystickMode: js });
    this.sendData(js ? ": SM JS" : ": SM BT");
    if (js) setTimeout(this.initJoystick, 200);
  },
  initJoystick() {
    wx.createSelectorQuery().select('#joystick-bg').boundingClientRect(res => {
      if (res) this.setData({ centerX: res.left + res.width/2, centerY: res.top + res.height/2,
joystickRadius: res.width/2 - 20 });
    }).exec();
  },
  stickStart(e) { this.stickMove(e); },
  stickMove(e) {
    if (!this.data.isConnected) return;
    const t = e.touches[0];
    let dx = t.pageX - this.data.centerX, dy = t.pageY - this.data.centerY;
    const dist = Math.sqrt(dx*dx + dy*dy), max = this.data.joystickRadius;
    if (dist > max) { const a = Math.atan2(dy, dx); dx = Math.cos(a)*max; dy = Math.sin(a)*max; }

    this.setData({ stickX: dx, stickY: dy });
    if (Date.now() - this.data.lastSendTime > 100) {
      this.sendData(`: V ${(dx/max).toFixed(4)} ${(-dy/max).toFixed(4)}`);
      this.data.lastSendTime = Date.now();
    }
  },
  stickEnd() { this.setData({stickX:0, stickY:0}); this.sendData(": V 0.0000 0.0000"); },

  // --- Camera & WebSocket ---
  openCamera() {
```

```
    const url = 'ws://192.168.137.178:8765';
    wx.showModal({ title: this.data.t.cam_title, content: url, editable: true, success: (res) => {
      if(res.confirm) this.startWebSocket(res.content || url);
    }});
  },
  startWebSocket(url) {
    this.setData({ isVideoMode: true, videoLoading: true });
    setTimeout(() => { this.initCanvas().then(() => this.connectSocket(url)); }, 200);
  },
  initCanvas() {
    return new Promise(resolve => {
      wx.createSelectorQuery().select('#videoCanvas').fields({node:true, size:true}).exec(res => {
        if (!res[0] || !res[0].node) return;
        const cvs = res[0].node, ctx = cvs.getContext('2d');
        const dpr = wx.getWindowInfo().pixelRatio;
        cvs.width = res[0].width * dpr; cvs.height = res[0].height * dpr; ctx.scale(dpr, dpr);
        this.canvasNode = cvs; this.canvasCtx = ctx;
        this.canvasWidth = res[0].width; this.canvasHeight = res[0].height;
        resolve();
      });
    });
  },
  connectSocket(url) {
    this.socketTask = wx.connectSocket({ url, success: () => console.log('WS Init') });
    this.socketTask.onOpen(() => { console.log('WS Connected'); this.setData({videoLoading: false}); });
    this.socketTask.onError((e) => { console.error('WS Error', e); this.closeCamera();
wx.showToast({title: this.data.t.disconnect, icon:'none'}); });

    this.socketTask.onMessage((res) => {
      const data = res.data;
      // 1. Photo Handling with robust cleaning
      if (typeof data === 'string' && data.startsWith("PHOTO:")) {
        const base64Data = data.replace("PHOTO:", "").replace(/[\r\n\s]/g, "");
        console.log("Photo received, size:", base64Data.length);
        this.saveBase64ImageToAlbum(base64Data);
      }
      // 2. Video Stream Handling
      else if (this.canvasNode && this.canvasCtx) {
        const img = this.canvasNode.createImage();
        img.onload = () => { if (this.canvasCtx) this.canvasCtx.drawImage(img, 0, 0, this.canvasWidth,
this.canvasHeight); };
        img.src = data.startsWith('data:image') ? data : '' + data;
      }
    });
```

```
  },
  closeCamera() {
    if (this.socketTask) { this.socketTask.close({ code: 1000, reason: 'User' }); this.socketTask =
null; }
    this.setData({ isVideoMode: false }); this.canvasNode = null; this.canvasCtx = null;
  },

  // --- PTZ & Camera Features ---
  startCamMove(e) {
    if (!this.data.isConnected && this.data.connectedDeviceId !== 'DEBUG') return;
    const dir = e.currentTarget.dataset.dir;
    this.updateCamAngle(dir);
    this.camTimer = setInterval(() => this.updateCamAngle(dir), 50);
  },
  stopCamMove() { if(this.camTimer) { clearInterval(this.camTimer); this.camTimer = null; } },
  updateCamAngle(dir) {
    let { camAngleLR: lr, camAngleUD: ud } = this.data, step = 1;
    if (dir === 'L') lr -= step; else if (dir === 'R') lr += step;
    else if (dir === 'U') ud += step; else if (dir === 'D') ud -= step;

    lr = Math.max(-90, Math.min(90, lr)); ud = Math.max(-60, Math.min(90, ud));

    // CRITICAL: Send negated values (-lr, -ud) as per logic requirements
    if (['L','R'].includes(dir) && lr !== this.data.camAngleLR) {
      this.setData({ camAngleLR: lr }); this.sendData(`: SVO LR ${-lr}`);
    } else if (['U','D'].includes(dir) && ud !== this.data.camAngleUD) {
      this.setData({ camAngleUD: ud }); this.sendData(`: SVO UD ${-ud}`);
    }
  },

  saveBase64ImageToAlbum(base64Data) {
    const fs = wx.getFileSystemManager();
    const filePath = `${wx.env.USER_DATA_PATH}/${Date.now()}.jpg`; // Must use .jpg

    console.log("Writing temp file:", filePath);
    fs.writeFile({
      filePath, data: base64Data, encoding: 'base64',
      success: () => {
        console.log("Write success, saving to album...");
        wx.saveImageToPhotosAlbum({
          filePath,
          success: () => {
            wx.showToast({ title: this.data.t.save_ok || 'Saved', icon: 'success' });
            fs.unlink({ filePath, fail: (e) => console.log('Del temp fail', e) });
```

```
          },
          fail: (err) => {
            console.error("Album save failed:", err);
            if (err.errMsg && (err.errMsg.includes("auth deny") || err.errMsg.includes("authorize:fail")))
{
              wx.showModal({
                title: this.data.t.auth_title || 'Auth Denied',
                content: this.data.t.auth_msg || 'Please enable album permissions in settings.',
                success: (res) => { if (res.confirm) wx.openSetting(); }
              });
            } else {
              wx.showToast({ title: `${this.data.t.save_fail || 'Failed'}: ${err.errMsg}`, icon:
'none' });
            }
          }
        });
      },
      fail: (err) => { console.error("Write temp failed:", err); wx.showToast({ title: 'Write Fail', icon:
'none' }); }
    });
  },

  takePhoto() { this.sendData(": CAM SHOT"); wx.vibrateShort(); wx.showToast({title: this.data.t.shot,
icon:'none'}); },
  toggleRecord() {
    const r = !this.data.isRecording;
    this.setData({ isRecording: r });
    this.sendData(r ? ": CAM REC" : ": CAM END");
    wx.showToast({ title: r ? this.data.t.rec_start : this.data.t.rec_end, icon:'none' });
  },
  toggleAI() { const s = !this.data.isAI; this.setData({ isAI: s }); this.sendData(s ? ": AI 1" : ": AI
0"); wx.vibrateShort(); },
  toggleTRK() { const s = !this.data.isTRK; this.setData({ isTRK: s }); this.sendData(s ? ": TRK 1" : ":
TRK 0"); wx.vibrateShort(); }
});
```